World Models in Action

3. Representation learning before generation

This chapter covers

- The assumptions built into self-supervised objectives
- Contrastive, bootstrap, self-distillation and redundancy-reduction objectives
- Masked prediction and the choice of prediction target
- Latent bottlenecks and the limits of representation evaluation

Chapter 2 established the contract for a useful state: it must preserve distinctions that change the future predictions the system needs to make, while discarding details that do not matter for those predictions. This chapter moves from that requirement to the learning problem: how can an encoder learn useful features when state labels are unavailable?

Let x denote an observation and let f_θ be an encoder with output $z = f_\theta(x)$. Self-supervised learning constructs a training signal from the data itself. A method may identify a matching view among candidates, predict a target produced by another branch, constrain feature statistics, or estimate content withheld from the input. In each case, the loss directly rewards a proxy relation; it does not directly optimize the full state contract from Chapter 2.

Methods are attributed to their original papers, and paper-reported findings are distinguished from results independently reproduced for this book. The short code excerpt is attributed to the official PyTorch implementation; it illustrates the stated calculation rather than presenting a new training implementation.

The question throughout this chapter is specific: what must an encoder preserve to make the chosen training objective easy to satisfy, and what evidence tells us whether that information is useful for a downstream prediction?

3.1. Self-supervised learning foundations

The central distinction

Self-supervised learning constructs a relationship from data and optimizes an encoder to satisfy it. That relationship may be useful without being sufficient for a world model. Separate the training target, the learned representation and evidence about downstream state quality.

3.1.1. A representation is not just a compressed observation

Let x be an observation and let f_θ be an encoder with parameters θ. Its output is a vector z:

$$z = f_\theta(x).$$

A smaller representation is useful only if the information removed is unnecessary for the task. A vector may preserve object identity while losing location or orientation. It may describe a frame's appearance while losing motion that is apparent only from a sequence. Compression can remove nuisance detail, but it can also merge observations whose futures differ.

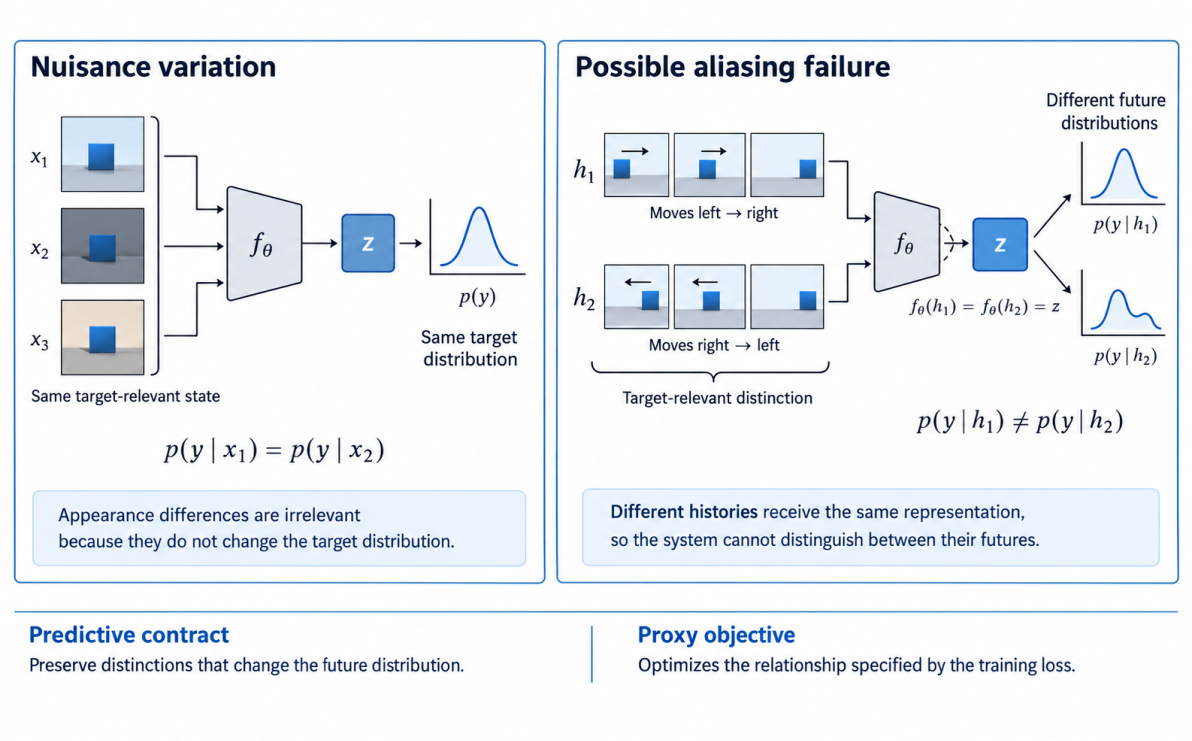

Figure 3.1 The representation contract. Nuisance appearance changes can share one representation when they leave the target distribution unchanged. Histories with opposite motion must remain distinct when their future distributions differ; merging them is a predictive aliasing failure.

3.1.2. Invariance is a decision

Suppose T_1 and T_2 are random transformations applied to the same input:

$$x_1 = T_1(x), \qquad x_2 = T_2(x).$$

A view-consistency objective encourages $f_\theta(x_1)$ and $f_\theta(x_2)$ to agree. The design decision is hidden in T_1 and T_2: they define which input differences the model is encouraged to ignore.

Random crops are useful when the cropped-out region is irrelevant to the intended representation. They are harmful when the missing region contains information needed to distinguish the scene or predict its next state. Color jitter can suppress nuisance lighting changes, but it can also remove a

visual cue on which a later prediction depends. The transformation is part of the learning problem, not a neutral preprocessing step.

An ideal invariance would identify observations that the downstream task truly treats as equivalent. A practical augmentation only approximates that relation. We should not assume that two views are interchangeable for every task just because they came from the same image.

The distinction can be written directly into the objective. Let T_1, T_2 be random transformations sampled from a chosen distribution and let d measure disagreement between two features. A consistency objective has the form

$$L_{\text{cons}}(\theta) = \mathbb{E}_{x,\, T_1,\, T_2}\big[d(f_\theta(T_1 x),\, f_\theta(T_2 x))\big].$$

The expectation averages over the views the augmentation pipeline actually creates, not every transformation that might occur in the world. If a transformation removes information, the encoder is rewarded for making the two resulting views agree despite that missing information. Information that varies unpredictably across views cannot reliably help reduce this loss. This is how augmentation defines an implicit preference over what the representation keeps.

This can be made precise. Let $\mathcal{T}$ be a set of transformations and define the orbit of an input x as $\mathcal{O}(x) = \{T(x) : T \in \mathcal{T}\}$. If an encoder is exactly invariant, then

$$f_\theta(Tx) = f_\theta(x) \qquad \text{for every} T \in \mathcal{T}.$$

All observations in the same orbit receive the same representation. The encoder therefore acts on equivalence classes of inputs rather than on every pixel-level distinction. This is the mathematical meaning of invariance: it removes distinctions that the chosen transformation family treats as irrelevant.

That reduction is beneficial only if the target of interest is constant within each class. If Y changes when x is transformed, no exactly invariant representation can preserve all information about Y. The problem is not that optimization failed; the objective has asked the encoder to discard the distinction. In real systems the equality is approximate, but the design pressure remains.

Invariance is also different from **equivariance**. An invariant representation remains unchanged under a transformation. An equivariant representation changes in a predictable way. For example, if an image is translated, an image-level category feature may ideally remain fixed, while a spatial feature map may shift in correspondence with the image. Classification and localization therefore need not prefer the same transformation behavior. A feature map that is deliberately equivariant can be more useful than a globally invariant vector when the target depends on where a feature occurs.

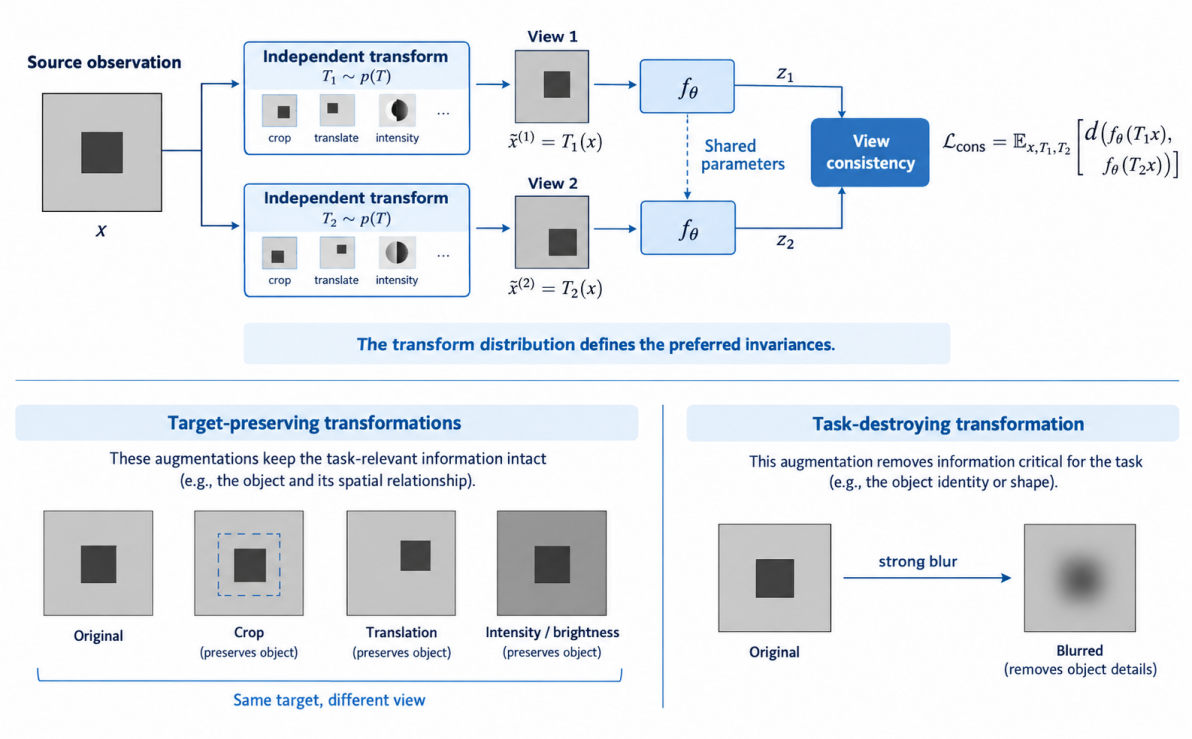

Figure 3.2 Augmentation defines the invariances rewarded by training. Two transformed views are trained to agree, so the augmentation distribution decides which changes the encoder learns to ignore. This helps only when those changes preserve information needed for the target.

Historically, visual self-supervision used hand-designed pretext tasks such as predicting image rotation (Gidaris et al., 2018), restoring the spatial order of jigsaw patches (Noroozi and Favaro, 2016), predicting spatial context (Doersch et al., 2015), and colorizing grayscale images (Zhang et al., 2016). These tasks assign synthetic targets to unlabeled data and make the target-design assumption easy to see. They remain useful conceptual examples, but the architecture survey below focuses on the broader contrastive, bootstrap, statistical, clustering and masked-prediction families that followed.

These are pretext tasks: targets are generated from the input rather than supplied by human annotation. They can teach useful features, but the target may admit shortcuts or require only a subset of the information needed downstream. Predicting image rotation, for example, need not preserve every scene distinction relevant to future prediction. Later objectives use relations between views, feature-batch statistics, or hidden content instead of one fixed synthetic label.

The principle behind representation learning

Every self-supervised method constructs a target relationship from the data. The relationship may be agreement between views, a ranking among candidates, a teacher’s output distribution, statistics of a feature batch, a cluster assignment or a missing patch. Ask what information makes that relationship easy to satisfy, then ask whether that information is

what the downstream model needs.

The methods in this chapter are grouped by their main teaching mechanism, not as a universal taxonomy. Mechanisms overlap: DINO matches teacher distributions over prototypes, iBOT combines masked patch prediction with teacher targets, and wav2vec 2.0 combines masking with contrastive discrimination. The section order is an editorial route through the material, not a claim that each method belongs to one exclusive family.

3.2. Contrastive learning

3.2.1. From pairs to candidate classification

A contrastive method begins with an anchor, a related positive example and distractors called negatives. The encoder maps each example to a feature vector. A similarity function scores the anchor against each candidate. Training rewards the positive for receiving a higher score than the negatives.

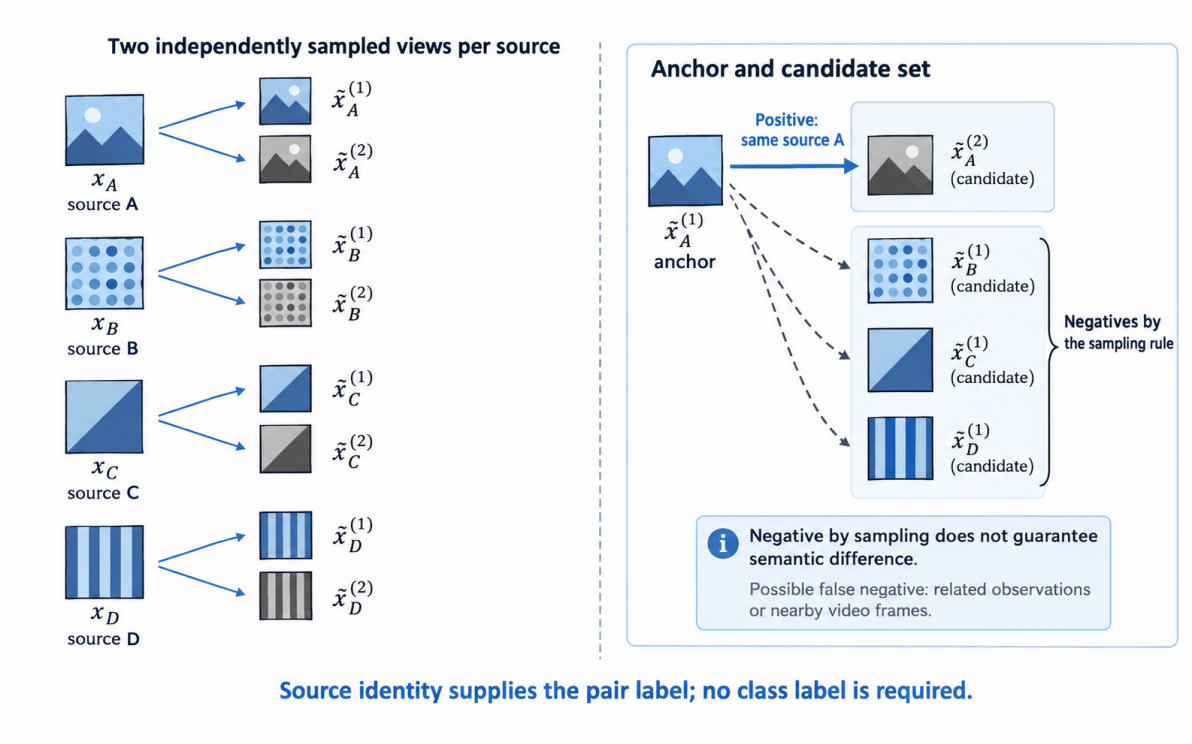

Figure 3.3 Constructing positives and negatives. The second augmented view of an anchor is the positive; views from other source examples are sampled negatives. Because labels follow source identity rather than semantic equivalence, related examples can become false negatives.

In the common two-view construction, each source example x_n is transformed independently twice:

$$\tilde{x}_n^{(1)} = T_n^{(1)}(x_n), \qquad \tilde{x}_n^{(2)} = T_n^{(2)}(x_n).$$

The two views of the same source form a positive pair. Views from other source examples form the negative candidates for that anchor. No class label is required; source identity supplies the pair label. The label therefore means “same source under two transformations,” not necessarily “same semantic class” or “same underlying state.”

If two different source examples are semantically equivalent, treating one as another’s negative creates a false negative relative to a semantic objective. This is not an error in cross-entropy. It is a mismatch between the labels implied by pair construction and the relation we ultimately want the representation to encode.

The negative set is part of the statistical problem, not just a source of extra examples. A contrastive loss trains the encoder to preserve distinctions that help identify the positive against the sampled candidates. If negatives are easy, the model can often solve the task using broad visual cues. As training improves, the most probable negatives dominate the gradient and the model is pushed to distinguish harder cases. This can sharpen useful features, but it can also make the learned geometry depend on details of the candidate distribution.

The loss does not tell the model which differences are semantically important. It only supplies a ranking label for the candidates. If the dataset repeatedly pairs two views of the same source image while treating every other image as negative, the representation is encouraged to keep same-source views close and other sampled views relatively separated. Two different images of the same object may still be pushed apart. Methods that use multiple positives, debiased negatives or semantic labels change this constraint, but they also change the learning problem.

For two feature vectors, cosine similarity is their dot product after unit normalization:

$$s(z_i, z_j) = \frac{z_i^\top z_j}{\| z_i \|_2 \, \| z_j \|_2}.$$

A temperature $\tau>0$ scales every similarity before softmax. If two candidates have similar scores, a lower temperature magnifies their difference in the logits and makes the probability distribution more concentrated. It does not simply make every gradient larger: when the positive already has probability close to one, its cross-entropy gradient is close to zero. Temperature changes both the probability assigned to each candidate and the factor relating a logit gradient to the underlying similarity gradient.

Suppose a batch contains N source images. Two stochastic views of each image give $2N$ encoded vectors. For anchor i, let $p(i)$ be the index of its other view. Excluding the anchor itself, every remaining view is a candidate. The NT-Xent loss for that anchor is

$$\ell_i = -\log \frac{\exp(s(z_i, z_{p(i)})/\tau)}{\sum_{a=1}^{2N} \mathbf{1}[a \neq i] \exp(s(z_i, z_a)/\tau)}.$$

The batch loss averages this quantity over all $2N$ anchors:

$$L_{\text{NT-Xent}} = \frac{1}{2N} \sum_{i=1}^{2N} \ell_i.$$

In the denominator of each term are one positive and $2N-2$ negatives. The positive is included in the denominator; the anchor is not. Stating the candidate set explicitly avoids ambiguity because papers do not all use the same symbol for the number of candidates or negatives.

This objective is ordinary softmax cross-entropy. Write $a_{ij} = s(z_i, z_j)/\tau$, and let q_{ij} be the softmax probability assigned to candidate j. Differentiating with respect to each candidate logit gives

$$\frac{\partial \ell_i}{\partial a_{ij}} = q_{ij} - \mathbf{1}[j = p(i)].$$

The positive logit receives gradient $q_{i,\,p(i)} - 1$, which pushes it upward under gradient descent. A negative logit receives gradient q_{ij}, which pushes it downward. The gradient on a negative is weighted by its current probability: a hard negative receives more repulsive gradient than an already dissimilar candidate. Since $a_{ij} = s(z_i, z_j)/\tau$, differentiation with respect to the unscaled similarity introduces the factor $1/\tau$.

For example, suppose one anchor has cosine similarities 0.80, 0.10 and -0.30 to its positive and two negatives, with $\tau = 0.5$. The logits are 1.6, 0.2 and -0.6. Softmax assigns the positive probability about 0.737, so the loss for this anchor is $-\log(0.737) \approx 0.306$. If all three candidates received equal probability, the loss would be $-\log(1/3) \approx 1.099$. The loss measures whether the correct candidate is easy to identify relative to the candidates that happened to be sampled.

For a cosine score $s(u, v) = \hat{u}^\top \hat{v}$, where $\hat{u} = u/\|u\|_2$ and $\hat{v} = v/\|v\|_2$, the derivative with respect to the first vector is

$$\frac{\partial s(u, v)}{\partial u} = \frac{1}{\|u\|_2} \left(\hat{v} - s(u, v)\hat{u}\right).$$

This gradient changes the direction of a normalized vector to improve its relative score. In a neural network, the same representation can appear as an anchor and as a candidate in other loss terms, so its total gradient accumulates contributions from all those roles. Contrastive learning is not merely “pull positives together”: it adjusts the representation so that positives outrank the negatives in the sampled candidate set.

Figure 3.4 combines the worked candidate-classification example with the derivative of the loss

with respect to each scaled similarity logit.

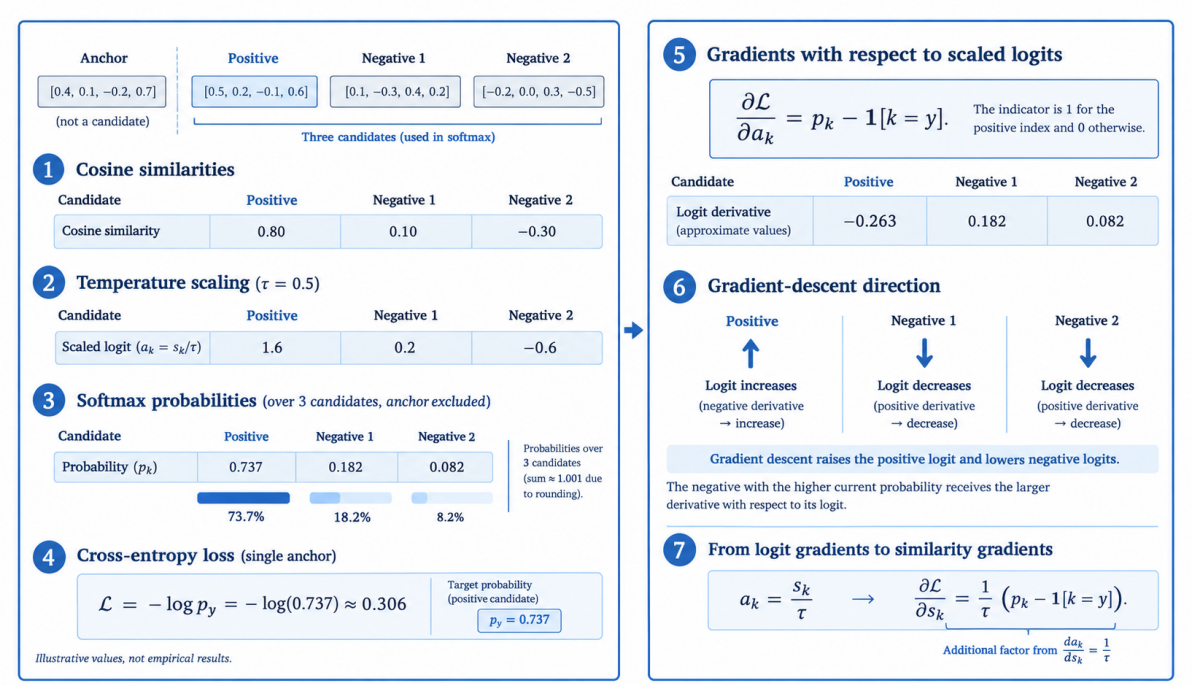

Figure 3.4 InfoNCE: Candidate probabilities and gradients. Temperature-scaled similarities become softmax probabilities and cross-entropy loss. Gradient descent raises the positive logit and lowers negative logits, with a harder negative receiving the larger update. Values are illustrative.

3.2.2. What InfoNCE does and does not prove

InfoNCE is a relative objective. The anchor must assign a higher score to its designated positive than to the sampled alternatives. Maximizing the positive similarity alone would not impose this ranking constraint. With an unnormalized dot product the model can also increase scores by increasing vector norms; cosine normalization bounds the score but does not remove dependence on the candidate-sampling protocol.

The positive label is supplied by data construction. In a two-view image objective, it means that two views came from the same source image. A negative can still depict the same semantic class as the anchor or show a nearly identical state. This is a false negative relative to a semantic objective, not an error in the cross-entropy arithmetic. The transformation distribution creates a related risk: if a view removes a relevant feature, consistency across those views can reward the encoder for ignoring it.

InfoNCE also has an information-theoretic interpretation, but only under a specified sampling construction. Suppose one index i is selected uniformly from K candidates; y_i is sampled from the conditional distribution $p(y \mid x)$, while the other $K-1$ candidates are sampled independently from the marginal $p(y)$. Bayes' rule gives

$$P(i \mid x, y_1, \ldots, y_K) = \frac{r(x, y_i)}{\sum_{j=1}^{K} r(x, y_j)}, \qquad r(x, y) = \frac{p(y \mid x)}{p(y)}.$$

The optimal classifier therefore ranks candidates by the density ratio: a pair receives a high score when it is more likely under the joint distribution than under independent sampling. A learned critic approximates this rule through its logits. The candidate-classification cross-entropy under this sampling construction yields the lower bound

$$I(X; Y) \geq \log K - \mathcal{L}_{\mathrm{NCE}},$$

where K includes the positive candidate. Equivalently, $\mathcal{L}_{\mathrm{NCE}} \geq \log K - I(X; Y)$. The finite candidate count limits this bound to $\log K$. The bound does not grow without limit merely because training continues. The softmax denominator normalizes only over sampled candidates; it is not generally the partition function of a normalized model over every possible image or latent value. This is the boundary between InfoNCE-style candidate classification and the broader energy-based framework (LeCun et al., 2006).

The assumptions matter. Nearby frames, repeated examples or multiple instances of one category can violate independent negative sampling. If negatives come from a different distribution, or candidates are dependent, the same bound does not automatically follow. The density-ratio view is related to noise-contrastive estimation, which also learns through a data-versus-noise discrimination problem (Gutmann and Hyvärinen, 2010), but the exact objective and guarantee depend on the sampling scheme. The density-ratio argument explains the role of the loss and its sampling assumptions; it does not turn an arbitrary contrastive loss into a certified measure of semantic information.

Finally, mutual information shared by two views is not the same as sufficiency for a later prediction. A feature can be shared between views yet irrelevant to the future quantity of interest. The information-theoretic bound says something about dependence under the sampling model, not whether all distinctions required by a downstream predictor have been retained.

3.2.3. Alignment and uniformity are different geometric requirements

For normalized representations, contrastive learning can be studied through two geometric diagnostics. **Alignment** asks matched views to be close. **Uniformity** asks independent examples not to crowd into a small part of the unit sphere. They explain different pressures in contrastive training, but they are not a universal exact decomposition of every self-supervised loss.

One alignment measure is

$$\mathcal{L}_{\mathrm{align}} = \mathbb{E}_{(x,\, x^+) \sim p_{\mathrm{pos}}} \| f(x) - f(x^+) \|_2^{\alpha},$$

where $\alpha > 0$. A uniformity measure is

$$\mathcal{L}_{\text{uniform}} = \log \mathbb{E}_{x,\ y\ \sim\ p_{\text{data}}} \exp(-t \parallel f(x) - f(y) \parallel_2^2),$$

where $t>0$ and x,y are independent samples. Nearby embeddings contribute strongly to the exponential, so crowding raises the quantity. Wang and Isola's (2020) analysis uses normalized features and a specified sampling setup. These measures are a lens on contrastive learning, not permission to describe every SSL algorithm as optimizing the same two losses.

Uniformity is not semantic correctness: features can spread out while encoding background texture or another nuisance variable. Alignment can be excellent because an augmentation erased a task-critical distinction. A useful representation requires invariances to match the prediction problem.

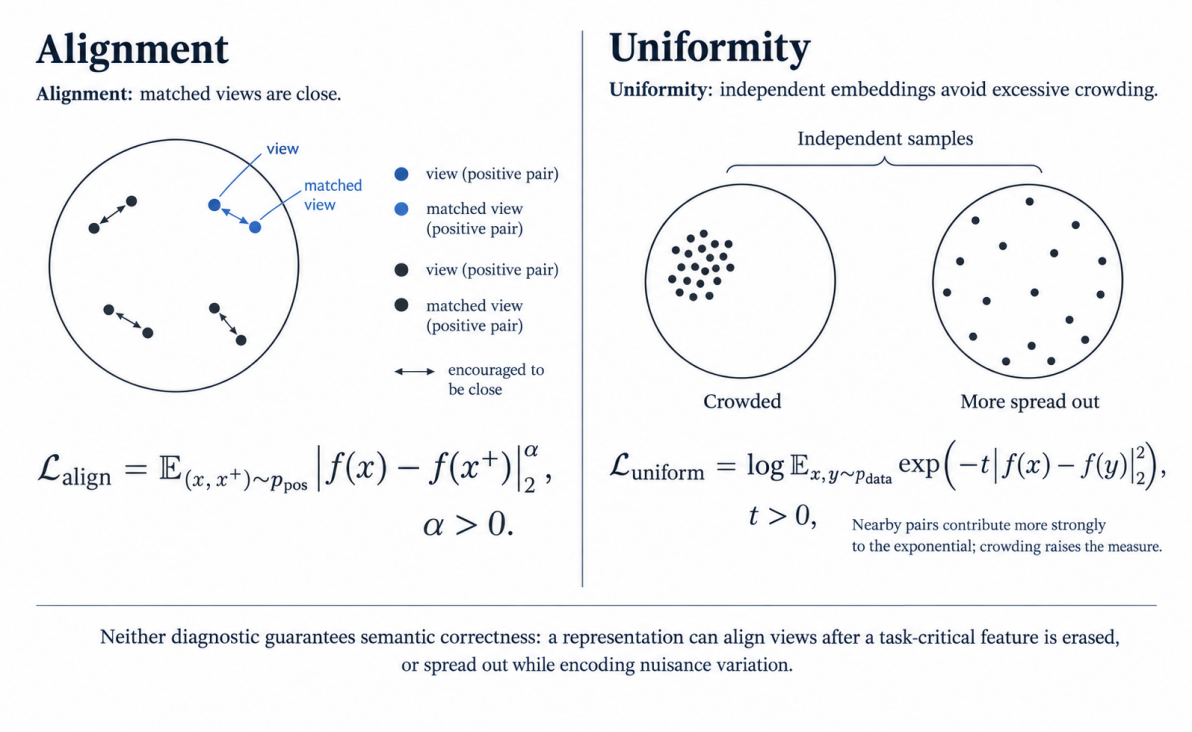

Figure 3.5 Alignment and uniformity are distinct geometric requirements. Alignment measures how closely matched views lie; uniformity measures whether independent embeddings crowd together on the normalized sphere. These diagnostics describe geometry, not the amount of task-relevant information retained.

3.2.4. SimCLR and MoCo: two ways to construct candidates

SimCLR (Chen et al., 2020) made the augmented-view recipe explicit. Two views pass through an encoder f and a nonlinear projection head g, and the objective is applied to $z = g(f(x))$. The representation before the head, $h = f(x)$, is commonly retained for downstream tasks. The projector allows the loss to shape a space specialized for the pretraining comparison, rather than requiring every property of that space to be optimal for later evaluation. This is a design choice supported by the SimCLR experiments, not a theorem that a projection head always improves every representation.

SimCLR's results depended on augmentation composition, batch size, architecture, training duration and optimization. A large batch increases the number of available negatives under an in-batch objective, but changes the classification problem itself and may add more false negatives. Its contribution is a clear recipe for contrastive visual learning, not evidence that every contrastive method needs one specific batch size or projector.

MoCo (He et al., 2020) uses a query encoder and a key encoder, with a queue storing keys from earlier minibatches. In the original recipe, the query and key features are L2-normalized before their dot products are scored. The query feature $q = f_{\theta_q}(x_q)$ is trained to identify its matching key $k^+ = f_{\theta_k}(x_k)$ among the positive and queued negative keys $k_j \in Q$. Its candidate loss has the form

$$\mathcal{L}_{\text{MoCo}} = -\log \frac{\exp(q^\top k^+ / \tau)}{\exp(q^\top k^+ / \tau) + \sum_{k_j \in Q} \exp(q^\top k_j / \tau)}.$$

The queue gives each query a larger candidate set than the current minibatch alone. The key branch receives no backpropagation gradient; its parameters follow an exponential moving average of the query encoder:

$$\theta_k \leftarrow m\theta_k + (1-m)\theta_q.$$

After the loss is evaluated against the current queue, keys from the current minibatch are enqueued and the oldest keys are dequeued to keep the queue length fixed. The queue therefore contains features produced by earlier parameter values. They become stale as training proceeds; a slowly moving key encoder reduces abrupt drift in their representation space but cannot make historical keys identical to current ones. This EMA update makes the key parameters a smoothed history of the query parameters: for fixed m, contributions from earlier updates decay geometrically. MoCo v2 (Chen et al., 2020) strengthened v1's augmentation, projection-head and optimization recipe while retaining the queue and momentum key encoder. The MoCo v1/v2 loss above therefore uses positive-plus-queue candidates. MoCo v3 (Chen et al., 2021) removes this persistent FIFO queue. Its reference implementation compares the online prediction for one crop against momentum-encoder targets for the other crop, gathered across workers. With Q_1, Q_2 denoting normalized online predictor outputs, K_1, K_2 the corresponding normalized momentum targets gathered across workers, and y the matching sample indices, its symmetric batch classification can be written

$$\mathcal{L}_{\text{v3}} = \text{CE}\left(\frac{Q_1 K_2^\top}{\tau}, y\right) + \text{CE}\left(\frac{Q_2 K_1^\top}{\tau}, y\right).$$

The momentum encoder remains stop-gradient and EMA-updated; the online path has a predictor. This is not the v1/v2 queue algorithm with a different backbone. The reference code also multiplies each cross-entropy term by a temperature-dependent constant, a loss-scale convention rather than a different candidate relation.

Both methods remain sensitive to the definition of positive pairs and negatives. The queue expands the candidate set, while the momentum encoder mitigates abrupt feature drift without eliminating staleness. Neither mechanism establishes that every negative is semantically different.

Figure 3.6 shows the queue-based MoCo v1/v2 mechanism. MoCo v3 changes that candidate construction and is distinguished in the text rather than crowded into the same drawing.

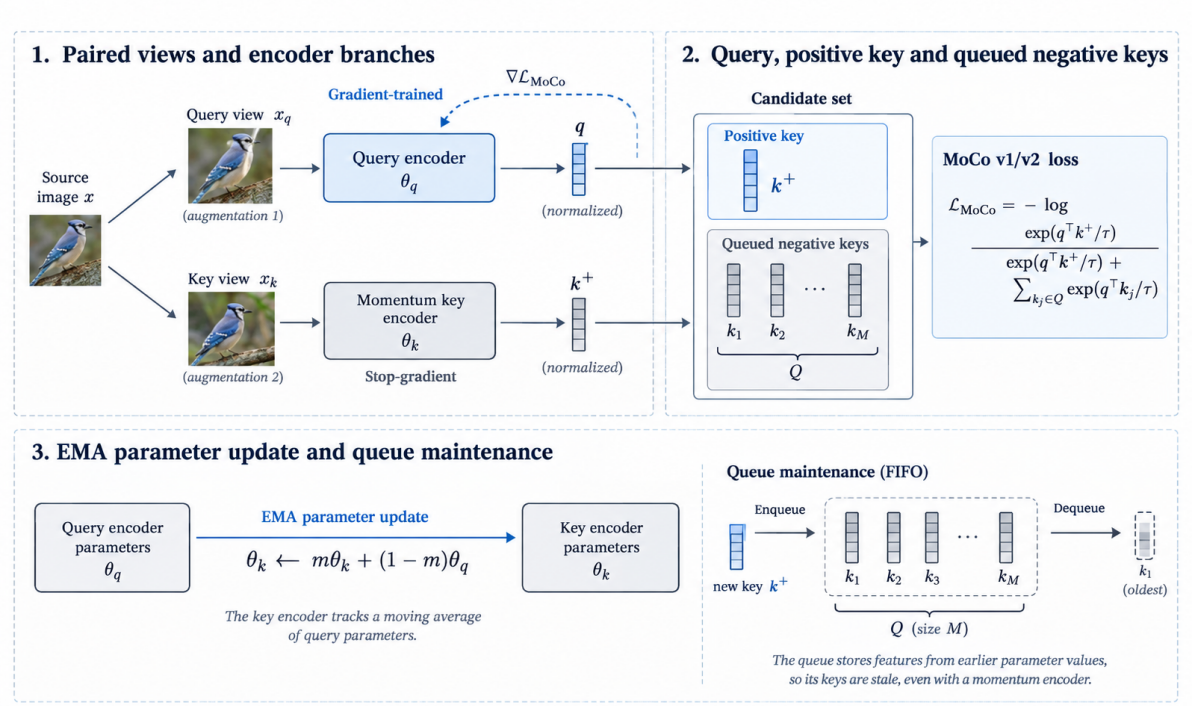

Figure 3.6 Momentum contrast: Query encoder, key encoder and queue. The query encoder receives contrastive gradients; a momentum-updated key encoder supplies the positive key and queue entries. The FIFO queue expands the candidate set, but stored features may become stale as the encoder changes.

3.2.5. Historical contrastive branches

SimCLR and MoCo are central modern examples, but their shared candidate-classification structure has earlier branches. These are not simply a sequence of equivalent losses.

Non-parametric instance discrimination (NPID) treats each image instance as a proxy class and uses a memory bank of features with a noise-contrastive objective. The signal is instance identity, which need not equal semantic identity.

PIRL asks representations to remain compatible across a pretext transformation while preserving an instance-discrimination signal. It changes the positive relation by bridging a transformed view and the original instance. The transformation helps only when the induced invariance matches downstream needs.

Contrastive Multiview Coding (CMC) contrasts features from different views or modalities of

the same source. The positive relation is a shared source rather than merely two augmentations of the same image tensor; usefulness depends on information shared between views.

AMDIM uses local and global representations at multiple feature scales. It contrasts a local feature with a global feature from the same image while mismatched images supply negatives. This introduces spatially indexed dependencies that global instance discrimination may not enforce.

These methods all construct a discrimination problem, but their positive relations and sampling distributions differ. Wu et al. (2018) study non-parametric instance discrimination; Misra and van der Maaten (2020) construct pretext-invariant representations; Tian et al. (2020) use contrastive multiview coding; and Bachman et al. (2019) introduce AMDIM. Any information-bound interpretation must follow the actual candidate sampler, not the generic label “contrastive.”

3.2.6. CPC as a historical bridge

Contrastive Predictive Coding (CPC; van den Oord et al., 2018) encodes a sequence and uses a context vector from the past to score future latent representations. An InfoNCE-style objective teaches the context to identify the observed future latent among candidate latents. CPC is a useful historical bridge between contrastive discrimination and predictive representations, but it is not the centerpiece of a contemporary visual SSL survey.

A CPC future-latent retrieval score only shows that the correct future code can be identified among the sampled candidates under the chosen objective. It does not by itself establish accurate multi-step rollout, a sufficient predictive state, or that errors remain controlled when predictions are fed back into the model.

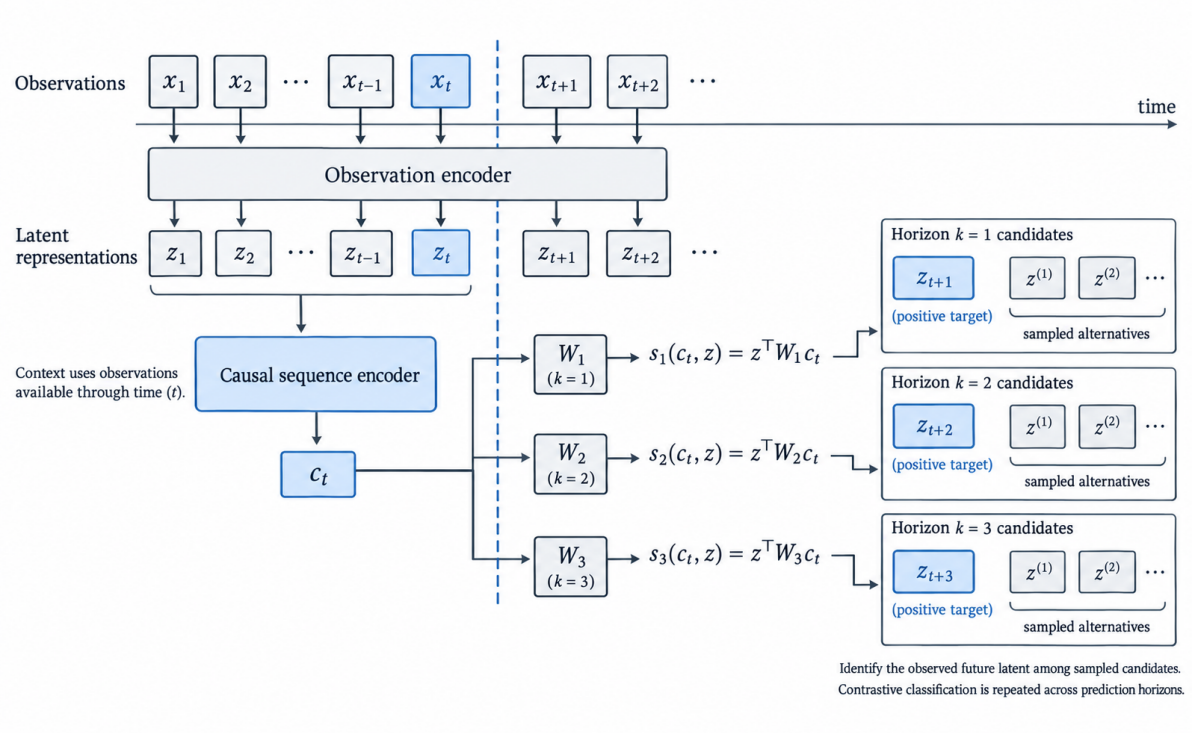

Figure 3.7 CPC: Causal context predicts future latent candidates. Past observations form a causal context that scores future latents at several horizons against sampled candidates. Future targets are used for comparison, not context input; retrieval success alone does not demonstrate recursive rollout or planning.

3.3. Non-contrastive objectives

Contrastive learning makes a designated positive score higher than sampled negatives. Non-contrastive methods remove that explicit competition, but they still need a mechanism that prevents agreement from becoming trivial. BYOL and SimSiam use asymmetric prediction pathways; DINO matches teacher distributions; Barlow Twins and VICReg constrain batch statistics.

3.3.1. Why agreement alone admits collapse

Suppose two views of the same input are trained only to produce equal representations. A simple invariance loss is

$$L_{\text{inv}} = \frac{1}{N} \sum_{i=1}^{N} \| z_i - z_i' \|_2^2 .$$

If the encoder maps every input to the same constant vector c, then $z_i = z_i' = c$ for every example and the loss is zero. The objective has no term requiring different inputs to remain distinguishable. A constant output is therefore a collapsed solution, even though it is useless for representing the data.

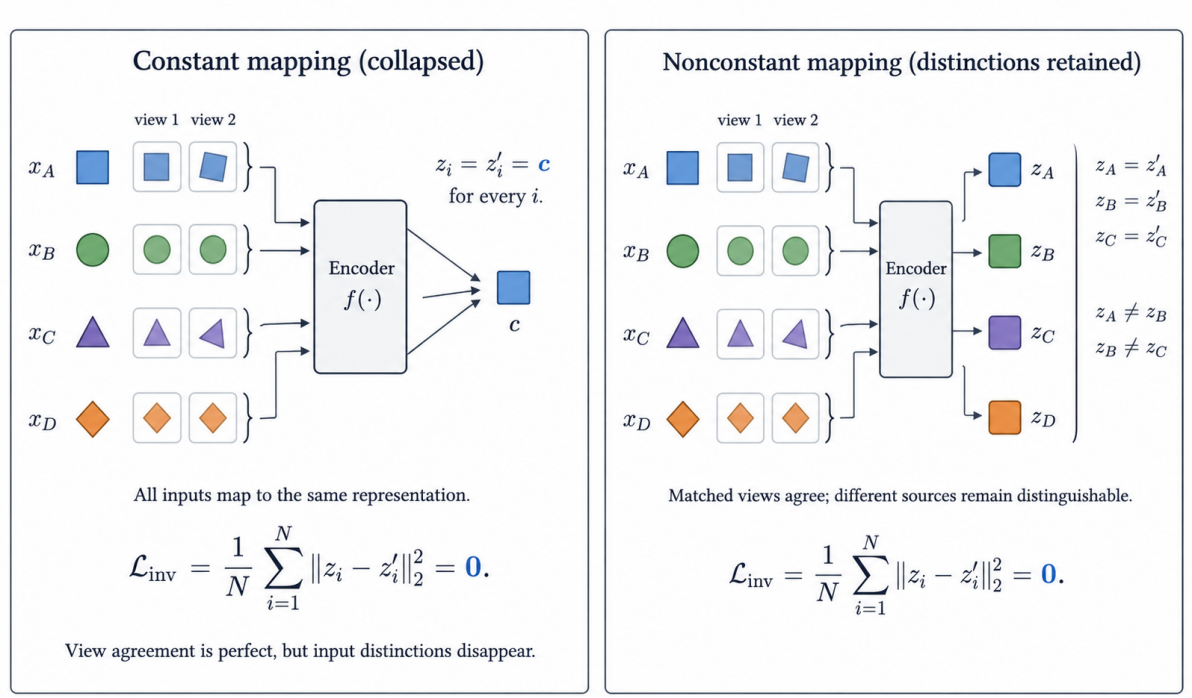

Figure 3.8 Why agreement alone permits collapse. Mapping every input to one constant vector gives perfect agreement while erasing differences. A nonconstant encoder can also achieve zero agreement loss, so agreement alone cannot exclude collapse; target updates and other constraints matter.

This observation does not mean every method that matches two views will collapse. It means agreement alone cannot rule collapse out. The architecture, stop-gradient placement, target updates, normalizations and optimization dynamics affect which solutions training reaches. The methods below differ in precisely those mechanisms; none should be described as having a universal guarantee based only on its loss formula.

It is useful to distinguish complete collapse from dimensional collapse. In complete collapse, all inputs map to the same vector and the representation contains no information about differences between examples. In dimensional collapse, the output varies, but most of its covariance is concentrated in a small number of directions. A model can therefore avoid the constant solution and still use its nominal d-dimensional space inefficiently.

These cases require different diagnostics. Per-coordinate variance detects a constant dimension; the spectrum of the covariance matrix shows whether variation is spread across many directions. But neither diagnostic answers whether those directions contain the information of interest. A feature can vary strongly with background texture and have high effective rank while carrying little information about the target. Anti-collapse is necessary for useful representations, not sufficient.

The covariance spectrum gives a more quantitative description of dimensional collapse. Let

$\lambda_1, \ldots, \lambda_d$ be the nonnegative eigenvalues of the feature covariance matrix. Normalize them into proportions

$$p_i = \frac{\lambda_i}{\sum_j \lambda_j}, \qquad r_{\text{eff}} = \exp\left(-\sum_i p_i \log p_i\right).$$

The quantity r_{eff} is an effective-rank diagnostic. If all variance lies in one direction, the distribution p_i is concentrated and the effective rank is near one. If variance is spread evenly across d directions, the effective rank is d. This statistic differs from the count of nonzero dimensions: a representation can have a formally full-rank covariance matrix while almost all its variance lies in a few directions.

Effective rank remains a geometric diagnostic, not a semantic one. A high-rank representation can encode color noise, background texture or image identity; a low-rank representation can still preserve all information required for a simple target. Variance, covariance and effective rank tell us about the distribution of features across a batch. They do not identify what the features mean or prove that a selected future variable is recoverable. That requires a separate target-specific evaluation.

3.3.2. BYOL and SimSiam: bootstrap prediction

BYOL (Grill et al., 2020) has an online encoder, a projection head and a predictor, alongside a target encoder and target projection head. Only the online network receives gradients. The target network is updated by an exponential moving average (EMA) of the online encoder and projector; it is not updated by backpropagation.

Let $z_1 = g_\phi(f_\theta(x_1))$, $p_1 = q_\psi(z_1)$, and $\bar{z}_2 = g_{\bar{\phi}}(f_{\bar{\theta}}(x_2))$. The prediction loss for one direction is the normalized regression

$$D(p_1, \text{sg}(\bar{z}_2)) = 2 - 2\frac{p_1^\top \bar{z}_2}{\| p_1 \|_2 \, \| \bar{z}_2 \|_2},$$

where sg denotes stop-gradient. A second term swaps the views. The online predictor is trained to match the target feature, while the target feature is treated as fixed for that gradient calculation.

The target parameters change according to

$$\bar{\theta} \leftarrow m\bar{\theta} + (1-m)\theta, \qquad \bar{\phi} \leftarrow m\bar{\phi} + (1-m)\phi,$$

where m is the momentum coefficient, usually close to one.

For clarity, the two-view objective can be written

$$L_{\text{BYOL}} = D(p_1, \text{sg}(\bar{z}_2)) + D(p_2, \text{sg}(\bar{z}_1)).$$

During a gradient step, the first term updates the online encoder, projector and predictor through p_1 while the target value $\bar{z}_2$ is fixed. The second term swaps the views. After the gradient step, EMA moves the target toward the updated online parameters. Across repeated updates, older online weights have geometrically decreasing influence; at $m = 0$ the target copies the online network, while larger m slows its change. This temporal smoothing is part of BYOL's optimization dynamics, not a universal proof against collapse.

SimSiam (Chen and He, 2021) uses the same encoder and projector for both views and a predictor on each branch, but no EMA target network. With $z_i = g(f(x_i))$ and $p_i = q(z_i)$, its symmetric loss has the form

$$L_{\text{SimSiam}} = D_{\text{SimSiam}}(p_1, \text{sg}(z_2)) + D_{\text{SimSiam}}(p_2, \text{sg}(z_1)),$$

where D_{SimSiam} is negative cosine similarity. The stop-gradient leaves a target's forward value unchanged but removes the gradient path through that target in the corresponding term. Because the loss is symmetric, each shared branch acts as predictor in one term and target in the other. BYOL instead maintains a separate target network whose parameters follow the EMA update.

The predictor and stop-gradient change optimization dynamics, but the constant-output solution remains a solution of the similarity objective. The distance used in the symmetric objective is

$$D_{\text{SimSiam}}(p, z) = -\frac{p^\top z}{\| p \|_2 \, \| z \|_2}.$$

Minimizing this negative cosine similarity maximizes directional agreement without requiring the two vectors to have the same norm. In $D(p_1, \text{sg}(z_2))$, the numerical value of z_2 still affects the loss, but its target-side gradient is removed. Because the loss is symmetric and both views share an encoder, that view still contributes encoder gradients when it serves as the predictor-side representation in the opposite term. The original SimSiam study found stop-gradient essential to its tested recipe; this is not a theorem that every detached Siamese network avoids collapse.

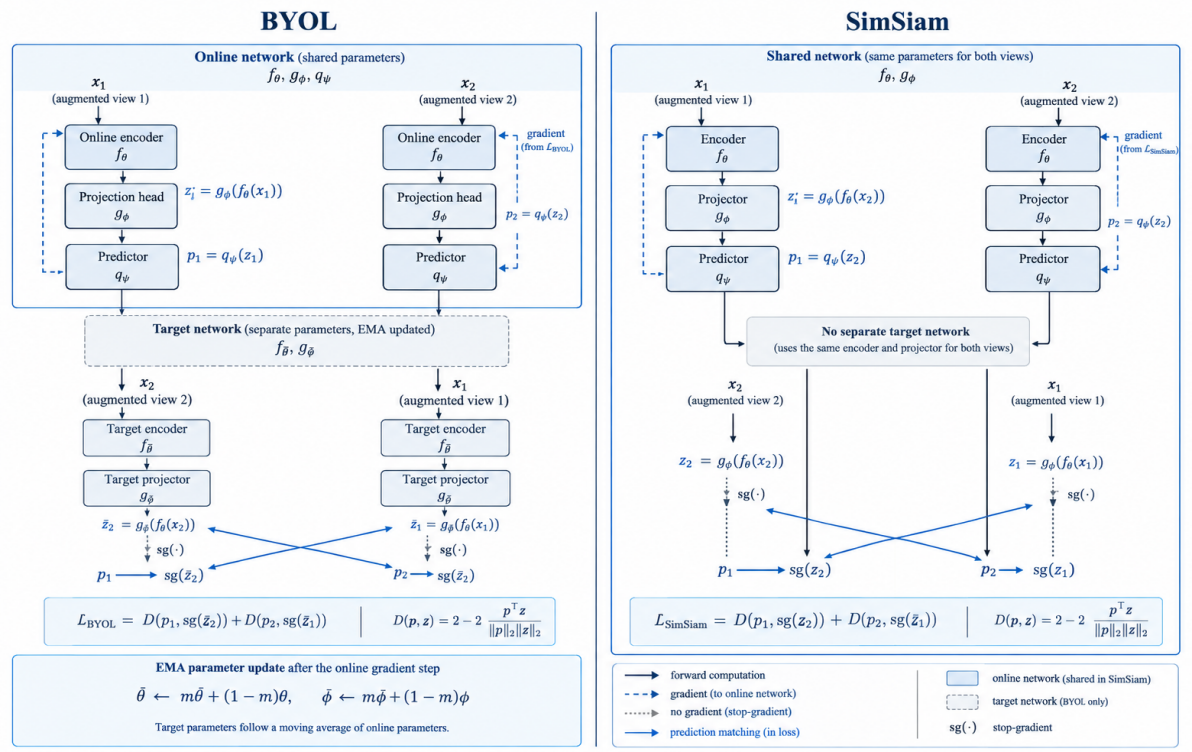

Figure 3.9 BYOL and SimSiam: Similar predictors, different target updates. Both predict a detached representation from another view. BYOL updates a separate target encoder by exponential moving average; SimSiam shares its encoder across views and uses stop-gradient without an EMA target network.

3.3.3. DINO: match a teacher's distribution

DINO (Caron et al., 2021) matches distributions rather than regressing one feature vector onto another. The student and teacher map different views to logits over learned prototype outputs. If $a_s(x)$ and $a_t(x)$ are their logits, the student and teacher distributions are

$$p_s(x) = \text{softmax}\left(\frac{a_s(x)}{T_s}\right), \qquad q_t(x) = \text{softmax}\left(\frac{a_t(x) - c}{T_t}\right).$$

Here c is a running center of teacher outputs, T_t controls teacher-target sharpness, and the teacher parameters follow an EMA of the student parameters. The student is trained across selected, different teacher-student views. For a target distribution q_t, the student's cross-entropy is

$$H(q_t, p_s) = -\sum_{k=1}^{K} q_{t,k} \log p_{s,k}.$$

Training sums this loss over selected teacher-student crop pairs.

Cross-entropy decomposes into the entropy of the target plus a divergence:

$$H(q_t, p_s) = H(q_t) + D_{\text{KL}}(q_t \,\|\, p_s).$$

For a fixed teacher target, the entropy term is constant with respect to the student, so minimizing cross-entropy makes the student distribution approach the teacher distribution. The decomposition also clarifies a collapse risk. If the teacher produces the same distribution for every input, the student can match it with a small KL divergence while learning little input-dependent structure. A low loss by itself does not show that the teacher's targets vary meaningfully across examples.

Centering and sharpening address different undesirable target distributions. Without centering, one output coordinate can dominate across many inputs, encouraging the teacher to repeatedly favor the same prototype. Centering counteracts this dominance, but by itself may encourage overly uniform outputs. Sharpening lowers the teacher temperature, making each target distribution more concentrated rather than uniform. Their combination helps retain input-dependent assignments. These are practical anti-collapse mechanisms of the DINO recipe; they should not be generalized into a theorem that centering and sharpening alone guarantee useful features in every architecture or training setup.

Prototype indices are learned output coordinates, not automatically human-defined categories. DINO is best understood as self-distillation with a prototype distribution, rather than as an ordinary clustering algorithm.

Multi-crop training supplies a useful extra constraint. The teacher receives the larger global crops, while the student also sees smaller local crops. A local crop usually lacks some of the global context, so its output must match a target produced from a different field of view. This encourages features to remain consistent across scales, but a local crop may not contain enough evidence to infer every property of the full scene. The crop protocol determines which consistency the objective rewards.

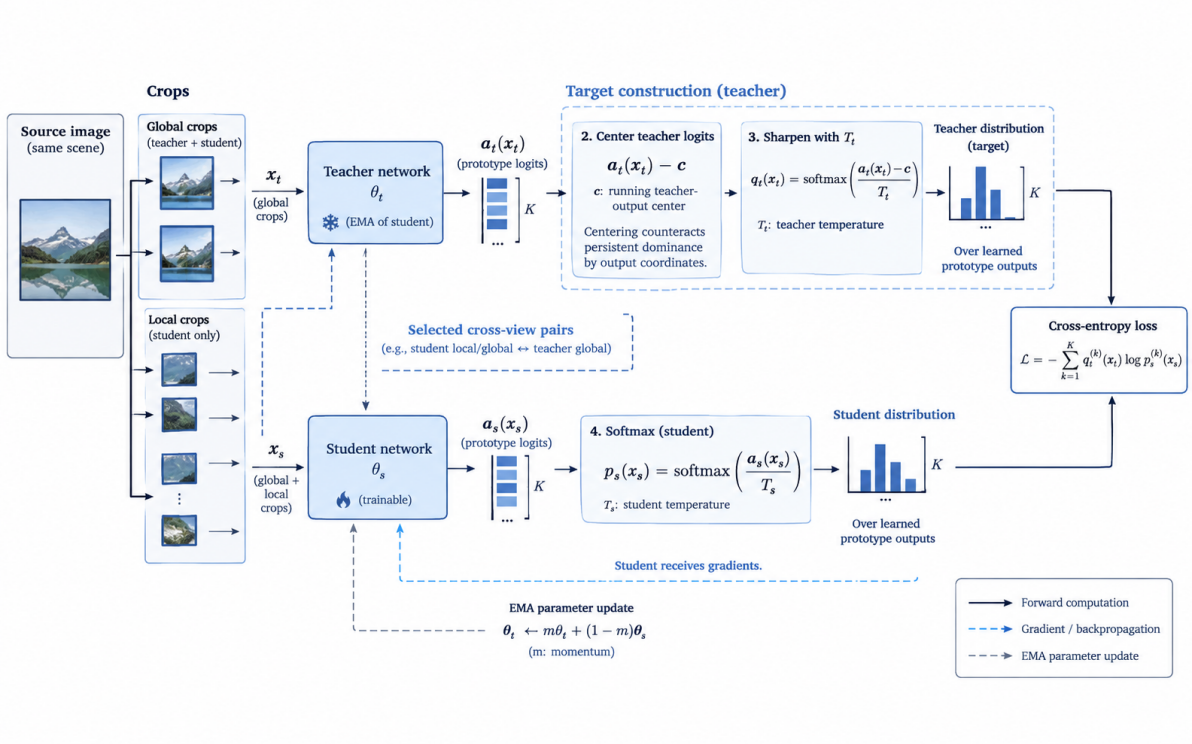

Figure 3.10 DINO teacher-student cross-view distillation. A student matches the teacher's prototype distribution for another view of the same source. Centering and sharpening shape the teacher target; the student receives gradients while the teacher tracks it through an exponential moving average.

These releases also illustrate a general caution: reported improvements belong to a complete training system, including data, architecture, optimization and evaluation, rather than to an isolated loss substitution. The following details separate the original DINO mechanism from components added in later recipes.

DINOv2 (Oquab et al., 2023) is a combined training recipe, not simply a replacement loss: it combines class-token DINO distillation, iBOT-style masked patch distillation and a KoLeo nearest-neighbour regularizer. The class branch matches teacher distributions across crops; the patch branch predicts teacher prototype distributions at masked student positions; KoLeo discourages local crowding in normalized backbone features. In the DINOv2 recipe, centering is used for class-token distillation, while Sinkhorn-Knopp balances patch-level assignments in the masked-patch branch. The precise target construction belongs to the cited recipe; these operations are not interchangeable.

DINOv3 (Siméoni et al., 2025) retains related distillation machinery and adds large-scale data, architecture and training choices. Gram anchoring, when enabled, matches teacher and student pairwise patch-similarity matrices rather than requiring coordinate-by-coordinate equality of feature vectors. It is a relational constraint, not proof of physical-state sufficiency, and it is configuration-dependent. The DINOv2 and DINOv3 papers describe broader training recipes; their results should not be attributed to Gram anchoring alone without controlled ablations.

3.3.4. Redundancy reduction and explicit variance constraints

BYOL, SimSiam and DINO make training dynamics or teacher targets central to avoiding trivial solutions. Barlow Twins (Zbontar et al., 2021) and VICReg (Bardes et al., 2022) express another idea: require paired views to agree, but constrain the statistics of features across a batch.

3.3.5. Barlow Twins: cross-correlation toward identity

Let Z^A and Z^B be feature batches from two views. For dimension i, let μ_i^A and σ_i^A be its mean and standard deviation across the batch, and likewise for view B. Standardize each dimension:

$$\tilde{z}^A_{b,\,i} = \frac{z^A_{b,\,i} - \mu_i^A}{\sigma_i^A + \epsilon}, \qquad \tilde{z}^B_{b,\,i} = \frac{z^B_{b,\,i} - \mu_i^B}{\sigma_i^B + \epsilon}.$$

Then define the cross-correlation matrix C:

$$C_{ij} = \frac{1}{N}\sum_{b=1}^{N} \tilde{z}^A_{b,\,i}\tilde{z}^B_{b,\,j}.$$

If corresponding dimensions agree and distinct dimensions are uncorrelated across views, C approaches the identity matrix. Barlow Twins minimizes the diagonal mismatch and penalizes off-diagonal cross-correlation:

$$L_{\mathrm{BT}} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2.$$

The diagonal terms encourage each coordinate to agree with its counterpart across views; the off-diagonal terms penalize linear cross-view correlation between different coordinates. Because features are standardized over the batch, these entries describe co-variation across examples rather than the relationship between two coordinates in one example. Duplicate coordinates incur an off-diagonal penalty, while poorly aligned coordinates incur a diagonal penalty.

The coefficient λ sets the strength of decorrelation relative to alignment. This is a second-order constraint: uncorrelated features can still have nonlinear dependence, and the coordinates are not automatically meaningful physical variables. Barlow Twins specifies a pattern for the batch cross-correlation matrix, not merely a distance between paired vectors.

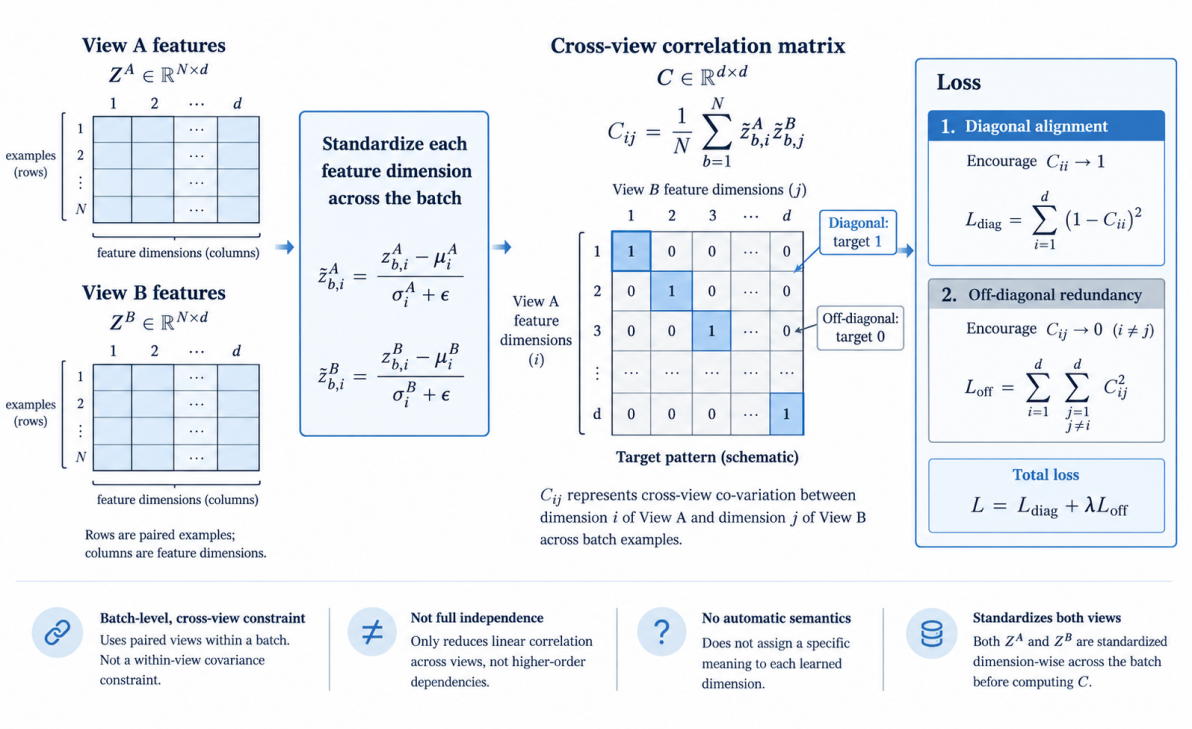

Figure 3.11 Barlow Twins: The cross-correlation matrix. The method standardizes paired-view features and drives their cross-correlation matrix toward identity: corresponding dimensions agree, while off-diagonal correlations shrink. This reduces linear redundancy but does not guarantee independence or semantic usefulness.

3.3.6. VICReg: make three constraints explicit

VICReg separates invariance, variance and covariance. For two feature batches Z and Z', the invariance term is a mean-squared difference:

$$s(Z, Z') = \frac{1}{N}\| Z - Z' \|_F^2 .$$

For each feature dimension j, let $\sigma_j(Z)$ be its standard deviation over the batch. A variance penalty puts a floor under each dimension's standard deviation:

$$v(Z) = \frac{1}{d}\sum_{j=1}^{d} \max\left(0, \gamma - \sqrt{\mathrm{Var}(Z_j) + \epsilon}\right).$$

The covariance term penalizes the squared off-diagonal entries of the empirical covariance matrix:

$$c(Z) = \frac{1}{d}\sum_{i\neq j} \mathrm{Cov}(Z)_{ij}^2.$$

The complete objective combines the three terms:

$$L_{\text{VICReg}} = \lambda s(Z, Z') + \mu[v(Z) + v(Z')] + \nu[c(Z) + c(Z')],$$

where λ, μ, ν set their relative weights. The invariance term aligns paired views. The variance term penalizes a feature dimension whose standard deviation falls below γ. The covariance term discourages linear redundancy between different dimensions. The original VICReg variance term uses an unsquared hinge. Once a feature's standard deviation reaches the margin, that dimension contributes zero variance penalty; the objective enforces a floor rather than rewarding ever-increasing variance.

The sample covariance used for $c(Z)$ is computed after centering the feature columns. For a batch $Z \in \mathbb{R}^{N \times d}$, let $\mu_j = N^{-1} \sum_{b=1}^{N} z_{b,j}$. Its covariance entries are

$$\mathrm{Cov}(Z)_{ij} = \frac{1}{N-1} \sum_{b=1}^{N} (z_{b,i} - \mu_i)(z_{b,j} - \mu_j).$$

The variance penalty and covariance penalty are deliberately different. A coordinate can have sufficient variance while duplicating another coordinate; covariance regularization penalizes that redundancy. A coordinate can have low covariance with every other coordinate yet be nearly constant; the variance penalty addresses that case. The invariance term then asks the two views to agree. These terms are complementary, not interchangeable.

The reductions are part of the definition. In $s(Z, Z')$, the squared feature-vector difference is summed over coordinates and averaged over examples; it is not automatically the same scalar as elementwise mean-squared error that divides by both batch size and feature dimension. That difference rescales the invariance term by the embedding dimension, changing the effective balance between λ, μ and ν. When comparing equations and implementations, check whether the norm is summed per example or averaged per coordinate before interpreting the coefficients.

A fully constant representation makes $s(Z, Z') = 0$ and makes all off-diagonal covariances zero, but it also makes each standard deviation zero. With the usual positive margin $\gamma > \sqrt{\epsilon}$, the variance term is therefore positive. In this sense, the variance floor makes exact constant collapse incompatible with minimizing all three terms simultaneously. This explicit penalty differs from BYOL or SimSiam, where a constant representation can make the view-agreement part of the objective small. It still does not guarantee that optimization never reaches a bad solution or that nonconstant directions encode the desired semantics.

The methods are related but not identical. Barlow Twins uses normalized cross-correlation as the central matrix objective. VICReg uses a direct invariance term, a standard-deviation floor and a covariance penalty. Their shared lesson is that view agreement and non-collapse are separate requirements. Their moment constraints are not a guarantee of a world-model-ready state.

A simple two-coordinate example makes the distinction concrete. If both coordinates are copies of the same varying scalar, their variances may exceed the margin, so the variance term is satisfied; however, their covariance is high and the covariance term penalizes them. If the second coordinate

is constant, its covariance with the first may be zero, but its standard deviation is below the margin and the variance term penalizes it. Only a feature batch with adequate spread and sufficiently low linear redundancy can make both statistical penalties small. The remaining variation can still describe the wrong thing, so a separate task-based evaluation is necessary.

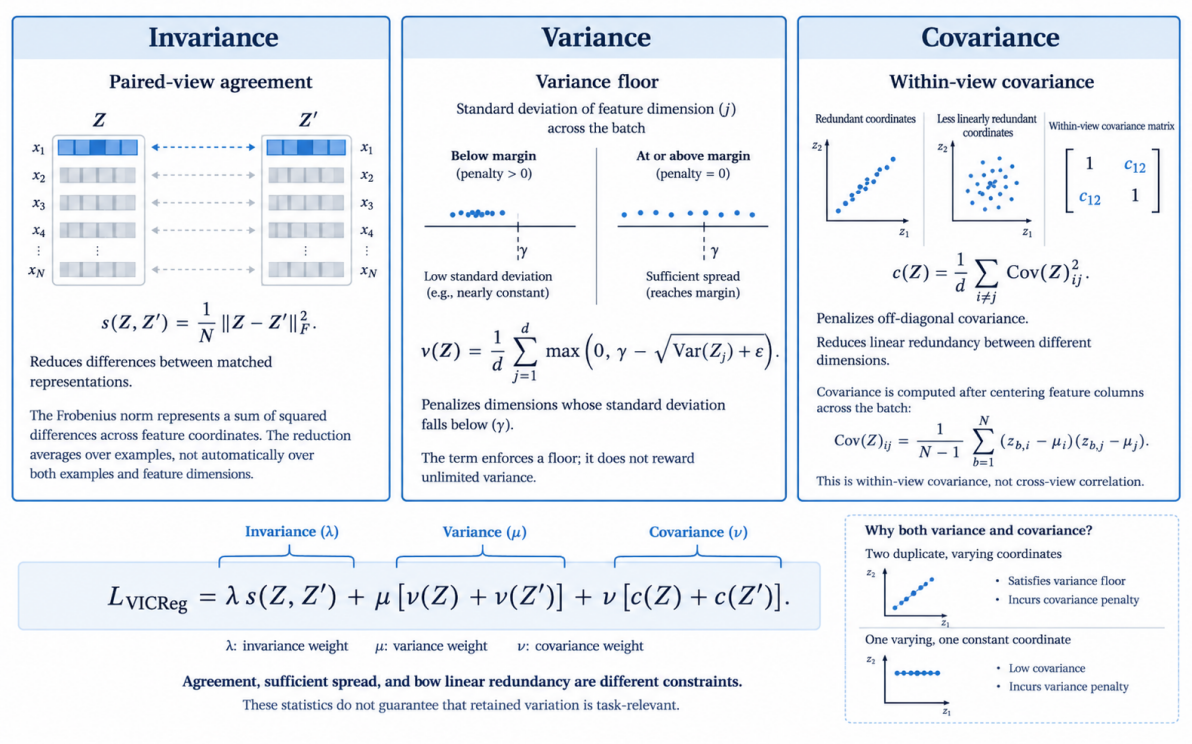

Figure 3.12 VICReg separates agreement, variance and covariance. The invariance term matches paired views, the variance term prevents constant dimensions, and covariance regularization discourages redundant directions. These constraints address distinct geometric failures but do not establish that retained variation is useful for the target.

Listing 3.1 Three VICReg terms in the official PyTorch implementation (source excerpt)

```
# Source: facebookresearch/vicreg/main_vicreg.py
repr_loss = F.mse_loss(x, y)

x = x - x.mean(dim=0)
y = y - y.mean(dim=0)
std_x = torch.sqrt(x.var(dim=0) + 0.0001)
std_y = torch.sqrt(y.var(dim=0) + 0.0001)
std_loss = (
    torch.mean(F.relu(1 - std_x)) / 2
    + torch.mean(F.relu(1 - std_y)) / 2
)

cov_x = (x.T @ x) / (self.args.batch_size - 1)
cov_y = (y.T @ y) / (self.args.batch_size - 1)
cov_loss = off_diagonal(cov_x).pow_(2).sum().div(self.num_features) + \
           off_diagonal(cov_y).pow_(2).sum().div(self.num_features)

loss = (self.args.sim_coeff * repr_loss
        + self.args.std_coeff * std_loss
        + self.args.cov_coeff * cov_loss)
```

Long expressions are wrapped to fit the book’s listing width without changing the calculation. The

official PyTorch implementation uses view agreement, a standard-deviation hinge with target one, and squared off-diagonal covariance. The helper and surrounding architecture are in the project's main training script. The upstream repository uses the MIT License; exact reductions and weights should be matched when reproducing a result.

JEPA-based predictive representation methods are covered in Chapter 4.

3.3.7. Other methods in brief

DeepCluster (Caron et al., 2018) alternates between clustering current features and training the encoder against the resulting pseudo-labels. SwAV (Caron et al., 2020) instead predicts one view's prototype assignment from another view's features, using balanced assignments in the referenced recipe. In both cases, clusters are training targets, not ground-truth semantic labels.

The same principle applies across modalities. CLIP (Radford et al., 2021) aligns images with paired text using a symmetric contrastive objective, while SigLIP (Zhai et al., 2023) uses pairwise sigmoid losses. BERT (Devlin et al., 2019) predicts masked tokens, GPT-style language models (Radford et al., 2019) predict the next token from a prefix, and ELECTRA (Clark et al., 2020) detects replaced tokens. In speech, wav2vec 2.0 (Baevski et al., 2020) contrasts contextual features with quantized targets, HuBERT (Hsu et al., 2021) predicts cluster-derived targets, and data2vec (Baevski et al., 2022) predicts teacher features. These are brief examples of different target constructions, not additional method tutorials. Generative modeling remains in the later chapters.

Table 3.1 Self-supervised objectives compared by the signal they construct

Representative methods	What the objective enforces	What it does not establish
SimCLR / MoCo	Identify a positive among sampled candidates	Negatives may be semantically related; features depend on candidate construction
CPC	Rank a future latent against sampled candidates	Candidate identification is not a calibrated transition model or an action-conditioned rollout
BYOL / SimSiam	Predict a detached view representation	Agreement alone does not rule out constant outputs
DINO family	Match teacher distributions over learned prototypes	Teacher targets can collapse or omit information of interest
Barlow Twins	Cross-view correlation matrix near identity	Second-order decorrelation is not semantic sufficiency
VICReg	View agreement plus variance and covariance constraints	Nonconstant, decorrelated features can still encode nuisance variation
MAE	Predict pixel values at masked patches	Pixel reconstruction does not prove predictive sufficiency
BEiT / iBOT / data2vec	Predict discrete codes or teacher features	The representation inherits the chosen target's biases
DeepCluster / SwAV	Predict or match pseudo-cluster assignments	Cluster assignments need not correspond to downstream concepts

3.4. Masked prediction: choose what the model must recover

3.4.1. One mask, several target spaces

Suppose an input is divided into visible positions V and masked positions M. An encoder processes the allowed context and a prediction head estimates a target y_i at masked positions:

$$L = \frac{1}{|M|} \sum_{i \in M} \ell(\hat{y}_i, y_i).$$

This form leaves the crucial choice open: what is the target? It might be pixel values, a discrete code, a word token, a replaced-token label or a contextualized feature. The mask tells the model where information is withheld; the target tells it what relationship must be learned.

For a probabilistic predictor, the general objective can be written as a conditional negative log-likelihood:

$$L(\theta) = -\mathbb{E}_{X_V,\, Y_M}[\log p_\theta(Y_M \mid X_V)],$$

where X_V is the visible context and Y_M is the hidden target. If Y_M is a discrete token or code, this becomes cross-entropy. If the model uses a Gaussian likelihood with fixed variance for a continuous target, minimizing negative log-likelihood is equivalent, up to constants and scale, to minimizing squared error. Under squared error, the optimal predictor in expectation is the conditional mean $\mathbb{E}(Y_M \mid X_V)$. Under cross-entropy, the optimal predicted class distribution is the conditional distribution of the target given the visible context.

The distinction appears directly in the loss. With squared-error regression, a predictor is rewarded for numerical proximity to a target vector. With cross-entropy, it is rewarded for assigning probability to the correct discrete label or teacher distribution. These losses define different geometries of error. For a discrete token, predicting a related code is still wrong unless the correct code receives probability; for a continuous target, errors are penalized according to their magnitude in the chosen feature space.

Masking also changes what the model is allowed to observe. A model trained to reconstruct a hidden patch from nearby visible pixels can exploit local texture continuity. A model trained to predict a contextualized teacher feature must reproduce a target shaped by the teacher’s wider context and representation space. Neither target is neutral: it specifies which aspects of the missing content distinguish a good prediction from a bad one.

The target-space schematic holds the mask fixed and changes only the prediction target.

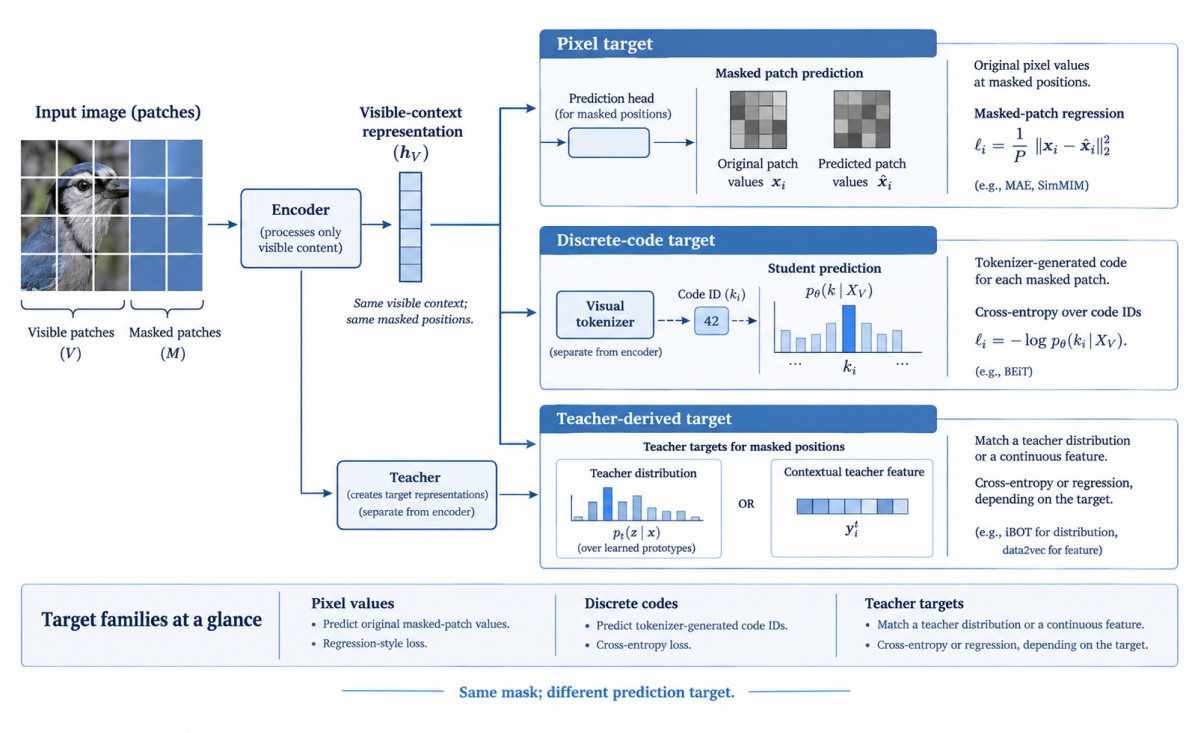

Figure 3.13 One visible context, several masked-prediction targets. The mask determines which positions are hidden; the target defines what must be predicted: pixel values, discrete tokenizer codes, or teacher-derived features or distributions. These are alternatives, not one architecture using every target.

3.4.2. MAE: predict masked pixels

A masked autoencoder (MAE; He et al., 2022) divides an image into patches and hides a large fraction of them. Only visible patches pass through the encoder. A smaller decoder receives the encoded visible patches and learned mask tokens inserted at the missing positions; it predicts pixel values at the original patch locations. The asymmetric design saves encoder computation because the encoder never processes the hidden tokens.

Let $m_i = 1$ when patch i is masked and 0 otherwise, with patch target x_i and prediction $\hat{x}_i$. Let P be the number of scalar pixel and channel values in one patch. The mean squared error for patch i is

$$\ell_i = \frac{1}{P} \| x_i - \hat{x}_i \|_2^2 .$$

The reconstruction objective averages this per-patch error over masked patches:

$$L_{\text{MAE}} = \frac{\sum_i m_i \ell_i}{\sum_i m_i}.$$

The normalization is first over the scalar values within each patch and then over masked patches, so visible patches do not contribute to the loss. Under squared error, the conditional mean is the best prediction in expected squared-error terms. If several patch contents are plausible from the

visible context, the conditional mean can blur those alternatives. Pixel reconstruction can nevertheless learn useful features; its limitation is that pixel accuracy does not guarantee that the retained encoder feature preserves every variable needed for a later prediction. The encoder, rather than the reconstruction decoder, is usually the component transferred to a downstream task.

The original MAE recipe uses a high masking ratio, 75% of image patches. The encoder therefore processes a sparse visible subset, reducing token-processing cost. The mask ratio also controls prediction difficulty: with few masked patches, more of the image is directly available as context; with too many, the visible content may not determine the missing patches well. The 75% setting is characteristic of the original recipe, not a universal optimum for every resolution, patch size, decoder or target.

Figure 3.14 shows why the MAE encoder sees only visible patches while its decoder reconstructs masked positions.

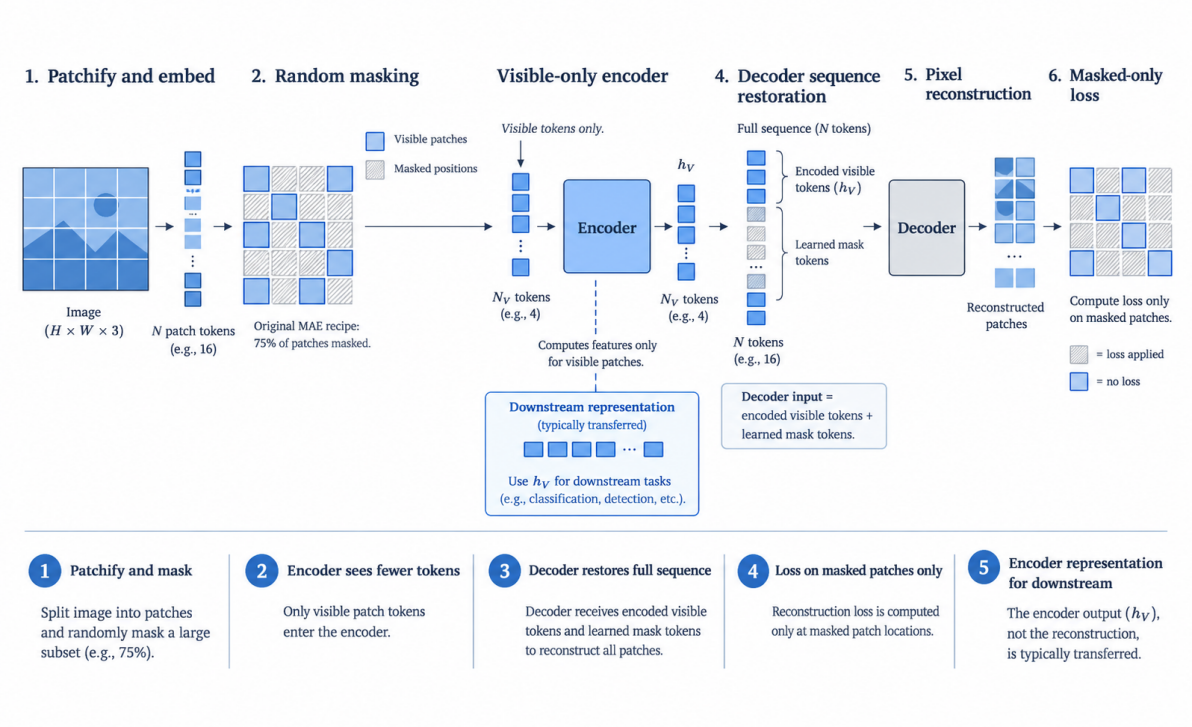

Figure 3.14 MAE's visible-only encoder and masked-patch decoder. Only visible patch tokens enter the encoder. The decoder restores the full sequence with mask tokens and predicts pixels for masked patches, reducing encoder computation at high mask ratios. Downstream tasks typically use the encoder features.

3.4.3. BEiT, iBOT and data2vec: change the target

BEiT (Bao et al., 2022) predicts discrete visual-token IDs produced by a learned tokenizer. Rather than regress pixel values, the model classifies the tokenizer's code for each masked patch. The tokenizer introduces its own prior: useful structure can be learned from the code vocabulary, but tokenizer errors and biases enter the target.

iBOT (Zhou et al., 2022) combines masked patch prediction with teacher-student self-distillation. SimMIM (Xie et al., 2022) uses pixel reconstruction; MaskFeat (Wei et al., 2022) predicts visual features such as HOG; BEiT v2 (Peng et al., 2022) improves the discrete-token route with a semantic visual tokenizer. These methods change the target while keeping the general idea of withholding input content. The target families can be written schematically as

$$L_{\text{BEiT}} = -\log p_\theta(k_i \mid x_{\text{visible}}), \qquad L_{\text{iBOT}} = -\sum_k q_{t,\,i,\,k} \log p_{s,\,i,\,k},$$

where k_i is the tokenizer's discrete code for a masked patch and $q_{t,\,i}$ is a teacher distribution over prototype outputs for patch i. BEiT assigns probability to a discrete target; iBOT matches a teacher distribution. For data2vec, a contextualized teacher vector y_i^t supplies a continuous target and the student predicts that vector under a regression loss such as

$$L_{\text{data2vec},\,i} = \| \hat{y}_i - y_i^t \|_2^2 .$$

These schematic expressions show the target-space distinction; each paper has further implementation choices, including the teacher, normalization, masking protocol and the subset of positions used in its loss. They are not full reproductions of those objectives. The differences reinforce the central point: the mask alone does not define the learning task.

These objectives predict a target **conditioned on what remains visible**. They do not, by themselves, specify a full generative model of the input distribution. A masked predictor may estimate a conditional mean, a discrete target distribution or a teacher feature. It need not define a normalized joint distribution over every possible image or support sampling a complete image. A generative model has additional requirements: its factorization or sampling process must define how a complete observation can be generated. This distinction prevents "reconstructs a patch" from being confused with "learns a generative model of images."

The target also controls the granularity of the error signal. Pixel regression distinguishes exact pixel values and penalizes large numerical deviations more than small ones. Discrete-code cross-entropy cares whether probability is assigned to the correct code, with similarities between codes not automatically reflected in the loss. Teacher-feature regression uses distances in a learned embedding space, so two different target patches may be considered close if the teacher represents them similarly. No target is universally best: the choice decides what counts as a meaningful prediction error.

data2vec (Baevski et al., 2022) predicts contextualized teacher features from masked input and was designed as a framework spanning speech, vision and language. A teacher feature has already aggregated context and training biases from the teacher; it is not simply the original pixel content transformed by a fixed operation. This can make the target more structured, but the student also inherits omissions and biases in the teacher's feature space.

BERT-style masked language modeling predicts selected corrupted tokens from bidirectional context. GPT-style causal language modeling predicts the next token from preceding tokens only. ELECTRA trains a discriminator to detect replaced tokens. These are brief analogies for target/context construction, not additional implementation tutorials.

Masked prediction and diffusion both learn from corrupted inputs, but their conditional tasks differ. A masked objective predicts selected hidden targets from visible context. A diffusion denoiser predicts a corruption-related target at a chosen noise level, and a specified reverse process turns those predictions into samples (Ho et al., 2020; Song et al., 2021). Neither patch reconstruction nor noise prediction alone establishes that an encoder is sufficient for a world-model state; diffusion mechanics belong in Chapter 9.

3.5. Latent bottlenecks

3.5.1. Dimension is capacity, not useful information

A latent bottleneck maps a high-dimensional observation or token grid into a constrained representation. The simplest form is a vector $h \in \mathbb{R}^d$. Lowering d limits the number of coordinates, but dimension alone does not measure useful information. A small vector may preserve the few variables needed for a prediction; a wider vector may devote many dimensions to texture, illumination or other nuisance variation.

The information-bottleneck principle expresses this trade-off using mutual information. Let X be the observation, Z its representation and Y a target relevant to the later task. A conceptual objective is

$$\min I(X; Z) - \beta I(Z; Y).$$

The first term penalizes information about the input retained by Z; the second rewards information about Y. The coefficient β controls the trade-off. This is a theoretical lens, not the loss explicitly optimized by every method in this chapter. Mutual information for deterministic continuous representations requires care; practical SSL objectives usually optimize a tractable proxy.

The information bottleneck gives a useful idealization of **minimal sufficiency**. If Z is a deterministic function of X and is sufficient for Y, then $Y \perp X \mid Z$, so

$$I(Z; Y) = I(X; Y).$$

The representation has retained all information that X provides about Y, even if it has discarded many details of X unrelated to Y. A minimal sufficient representation seeks to retain that predictive information while minimizing what it carries about the rest of the input. The information-bottleneck objective expresses a trade-off, not a direct recipe for finding a unique vector. Different representations can be equally sufficient, especially when features are transformed by invertible

maps.

The important question is not “how small can the vector be?” It is “which distinctions can be discarded without changing the predictions we need to make?” A representation useful for image retrieval may be insufficient for predicting motion if it merges observations with different velocities or hidden states.

This is why raw dimension is not the same as information rate. A real-valued vector with a small number of coordinates can, in principle, encode many distinctions in very fine numerical differences. In practice, finite precision, noise, limited data and the smoothness of learned functions constrain how reliably such distinctions can be used. Conversely, a very wide feature may carry highly redundant coordinates. Dimension is a proxy for capacity, while useful compression concerns which differences remain reliably decodable under the conditions where the model will be used.

3.5.2. The projection head and the feature that survives

Many contrastive and bootstrap methods use a projection head between an encoder and the objective:

$$h = f_\theta(x), \qquad z = g_\phi(h).$$

The loss is calculated on z, while the representation evaluated downstream is often h. A projection head gives the pretraining objective a dedicated space in which to impose its constraints. It does not follow that z and h have the same geometry or serve the same downstream purpose.

The exported tensor must be identified explicitly. Some evaluations use the feature h before the projector; others use intermediate or patch-level transformer features. A low pretraining loss measured on z is not itself evidence that the feature h has the same geometry or usefulness. The projector can change the geometry substantially because it is a learned, generally nonlinear map. The pretraining loss directly constrains z, not every pairwise relation in h. Claims about representation quality should state the tensor and evaluation protocol rather than infer quality from training loss alone.

3.5.3. A global vector or a grid of local features?

A global vector is compact and convenient for classification, retrieval and simple interfaces. A grid of patch features preserves approximate locations and correspondences. A temporal sequence preserves history only if its aggregation does not erase the changes needed by prediction. These forms are related but not interchangeable.

For patch features $h_1, \ldots, h_P$, simple average pooling forms

$$h_{\text{global}} = \frac{1}{P}\sum_{i=1}^{P} h_i.$$

This compresses the grid into one vector and removes explicit patch indexing from the output. Distinct spatial arrangements can therefore produce the same pooled vector. Retaining the grid preserves a feature for each patch and its token position, although the feature values themselves are not guaranteed to encode accurate geometry. The choice is a representational trade-off: a vector is compact, while a grid makes local correspondences accessible to later components. Global pooling is appropriate when spatial arrangement is irrelevant, but can be harmful when the target depends on location or correspondence between regions. A temporal representation also has to retain motion-related information when it changes the target future.

The pooling equation also gives a simple source of aliasing. Consider two feature grids whose patch vectors are identical as a multiset but appear in different spatial orders. Average pooling gives the same sum, hence the same global vector, even though the arrangements differ. If the target depends on the arrangement, the pooled representation cannot be sufficient for that target. Positional information elsewhere in the encoder can reduce such collisions, but the pooling operation itself does not preserve the index of each patch.

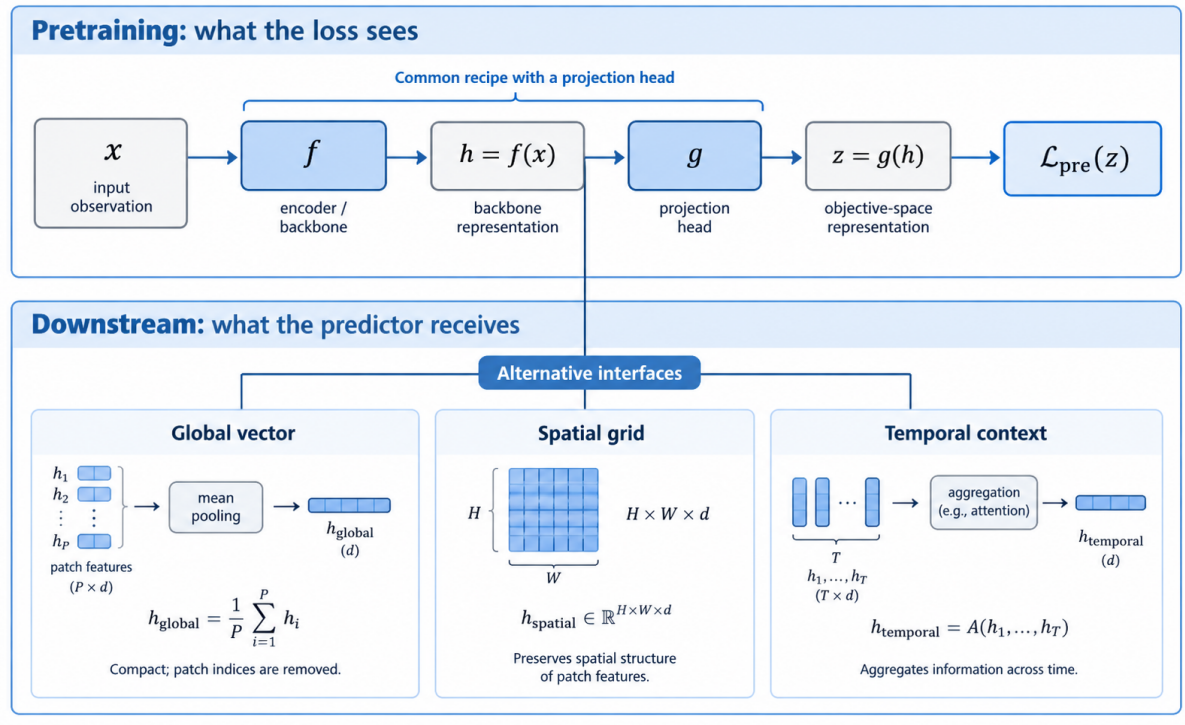

Figure 3.15 The training representation and the downstream interface. Pretraining may optimize the projector output, while downstream tasks often use backbone features. Those features can be exported as a pooled vector, spatial grid or temporal context; the interface determines which distinctions remain accessible.

3.5.4. Compression and predictive sufficiency

A useful representation retains enough information for the relevant prediction while suppressing nuisance variation. There is no universally optimal latent dimension for all tasks. In some settings a small state may be sufficient; in others, spatial features, temporal context or multiple latent components may be necessary.

Here, **compact** means constrained and usable by downstream prediction, not necessarily information-theoretically minimal. The bottleneck should be selected and tested against the downstream task. A low-dimensional representation that aliases important states is not a success; a larger representation is not automatically better if it mostly stores nuisance detail.

3.6. Compact state representations

Chapter 2's state contract now becomes an evaluation question: does the representation preserve the distinctions needed to predict a specified target from the history available at inference? A low pretraining loss answers a narrower question. It shows that the model has learned to satisfy the relation supplied by its training objective.

A linear probe tests whether a selected property is accessible through a linear readout:

$$\hat{y} = Wz + b.$$

A high score establishes linear accessibility under the stated probe, data split and feature choice. A low score does not establish that the property is absent; it may be encoded nonlinearly or be inaccessible to the chosen readout. Report the exported tensor, normalization, readout, training split and held-out metric so that a probe result has a clear interpretation.

Other diagnostics answer different questions. Per-coordinate variance detects dimensions that are constant across a batch. The covariance spectrum measures how feature variation is distributed across directions. Nearest-neighbour retrieval tests local geometry under a selected metric, and reconstruction measures recovery of a chosen target. Each diagnostic has scope; none alone certifies that the representation preserves everything needed for future prediction.

Using Chapter 2's predictive-sufficiency condition, let X denote the observation or history available to the encoder, $Z = f(X)$ its representation, and Y the prediction target. Sufficiency for that target means

$$P(Y \mid X) = P(Y \mid Z).$$

This condition depends on the target and its horizon. A representation may be sufficient for one-step category prediction and insufficient for a precise future location. For action-conditioned prediction, the same action must be considered on both sides of the comparison.

Aliasing violates the condition when two histories map to the same representation but imply

different future distributions:

$$f(x_1) = f(x_2) = z, \qquad P(Y \mid x_1) \neq P(Y \mid x_2).$$

For example, two video histories can end with an object at the same image location while showing motion in opposite directions. If the representation merges those histories, a predictor that receives only z must return the same distribution for both. Increasing the predictor's capacity cannot recover a distinction the encoder discarded. This is a target-relative failure, not a claim that every pair of similar frames must have different futures.

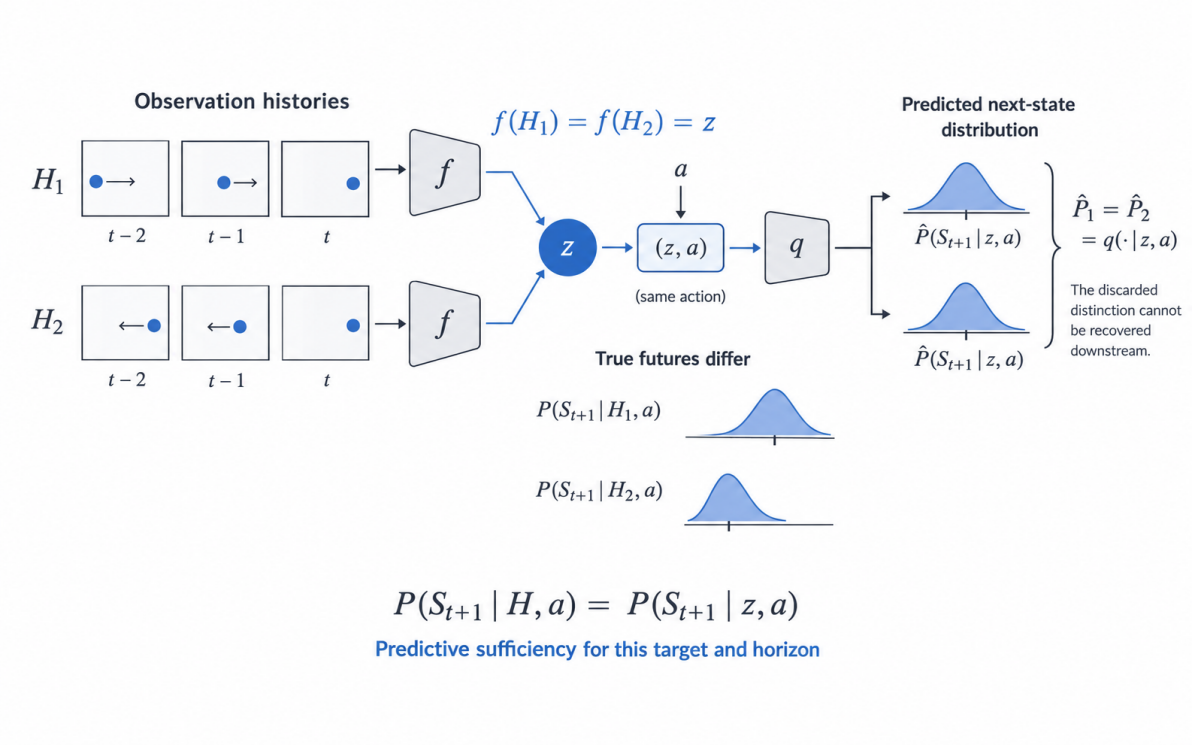

Figure 3.16 Aliasing is a failure of predictive sufficiency. Two histories map to the same representation but imply different next-state distributions under the same action. A predictor given only that representation and action must return the same distribution for both, so it cannot predict both correctly.

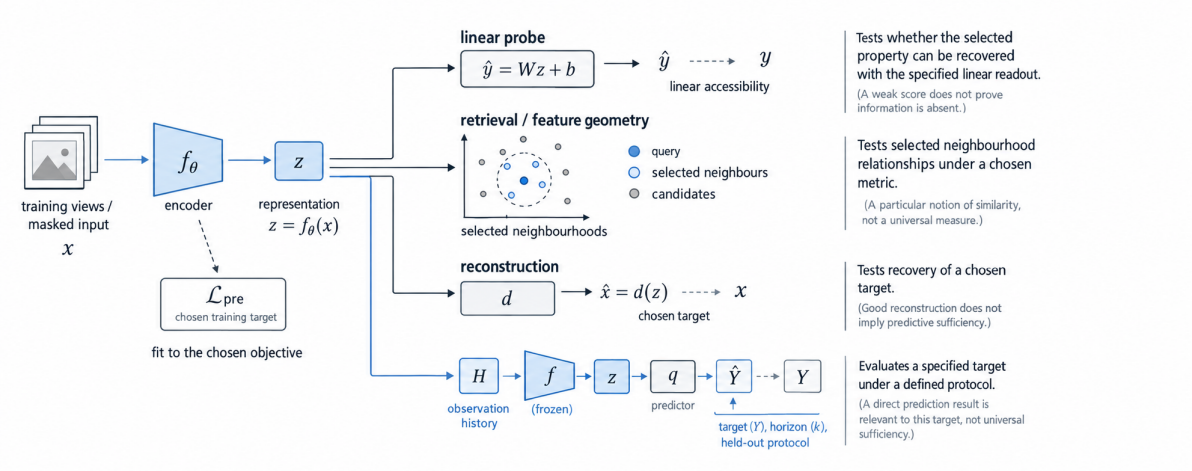

Figure 3.17 A training loss and a probe answer different questions. No single metric certifies predictive sufficiency. Pretraining loss measures fit to its chosen target; a probe tests accessibility under a specified readout, while sufficiency requires evaluating a named target and prediction horizon on held-out data.

3.7. Summary

Self-supervised learning constructs a target relationship from data. Contrastive methods such as SimCLR and MoCo rank positives against sampled candidates; CPC applies that idea to future latents. BYOL and SimSiam predict detached view targets but differ in how their targets change. DINO matches teacher distributions, Barlow Twins constrains cross-view correlations, and VICReg separates agreement, variance and covariance penalties. Clustering methods create pseudo-targets from current features.

Masked methods differ in whether they predict pixels, discrete codes or teacher features. Across these families, the target and loss specify which relationships training rewards, while the encoder architecture and output bottleneck determine where those constraints act. None of these ingredients, nor a low pretraining loss alone, establishes predictive sufficiency. That claim must be evaluated for a named target, input history and prediction horizon.

3.8. Exercises

1. For an anchor with one positive and $K-1$ negatives, derive the softmax-cross-entropy gradient with respect to each scaled similarity logit. Explain why a high-probability negative receives a larger gradient than a low-probability negative.
2. A frozen encoder is excellent at image retrieval but weak at predicting motion. Give two plausible reasons for the mismatch, then design a target-specific evaluation that distinguishes missing temporal information from information that is present but difficult for a linear readout to access.
3. State the parameter-update difference between BYOL and SimSiam. In each method, which pathway receives gradients, and which mechanism changes the target?

4. Write the Barlow Twins objective in terms of the cross-correlation matrix. Explain the role of its diagonal and off-diagonal terms, then contrast it with VICReg's three terms.
5. Compare the targets used by MAE, BEiT and iBOT/data2vec. For each method, state what is predicted and what the loss measures.
6. Give two observation histories x_1, x_2 that map to the same representation but have different conditional future distributions. Explain why no downstream predictor that receives only that representation can predict both distributions correctly.
7. Decompose DINO's teacher-student cross-entropy into target entropy and KL divergence. Explain what happens if the teacher returns the same distribution for every image.
8. For a batch whose second VICReg coordinate is constant but whose first coordinate varies, determine which term penalizes the second coordinate and why the covariance penalty alone would not be enough.

3.9. References

Bachman, P., Hjelm, R. D. and Buchwalter, W. (2019). Learning Representations by Maximizing Mutual Information Across Views. arXiv:1906.00910.

Baevski, A., Zhou, Y., Mohamed, A. and Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. **Advances in Neural Information Processing Systems**. arXiv:2006.11477.

Baevski, A., Hsu, W.-N., Xu, Q., Babu, A., Gu, J. and Auli, M. (2022). data2vec: A General Framework for Self-Supervised Learning in Speech, Vision and Language. **International Conference on Machine Learning**. arXiv:2202.03555.

Bao, H., Dong, L. and Wei, F. (2022). BEiT: BERT Pre-Training of Image Transformers. **International Conference on Learning Representations**. arXiv:2106.08254.

Bardes, A., Ponce, J. and LeCun, Y. (2022). VICReg: Variance-Invariance-Covariance Regularization For Self-Supervised Learning. **International Conference on Learning Representations**. arXiv:2105.04906.

Caron, M. et al. (2018). Deep Clustering for Unsupervised Learning of Visual Features. **European Conference on Computer Vision**. arXiv:1807.05520.

Caron, M. et al. (2020). Unsupervised Learning of Visual Features by Contrasting Cluster Assignments. **Advances in Neural Information Processing Systems**. arXiv:2006.09882.

Caron, M. et al. (2021). Emerging Properties in Self-Supervised Vision Transformers. **IEEE/CVF International Conference on Computer Vision**. arXiv:2104.14294.

Chen, T. et al. (2020). A Simple Framework for Contrastive Learning of Visual Representations. **International Conference on Machine Learning**. arXiv:2002.05709.

Chen, X. et al. (2020). Improved Baselines with Momentum Contrastive Learning. arXiv:2003.04297.

Chen, X. and He, K. (2021). Exploring Simple Siamese Representation Learning. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2011.10566.

Chen, X. et al. (2021). An Empirical Study of Training Self-Supervised Vision Transformers. arXiv:2104.02057.

Clark, K. et al. (2020). ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. **International Conference on Learning Representations**. arXiv:2003.10555.

Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. **North American Chapter of the Association for Computational Linguistics**. arXiv:1810.04805.

Doersch, C., Gupta, A. and Efros, A. A. (2015). Unsupervised Visual Representation Learning by Context Prediction. **IEEE International Conference on Computer Vision**. arXiv:1505.05192.

Gidaris, S., Singh, P. and Komodakis, N. (2018). Unsupervised Representation Learning by Predicting Image Rotations. **International Conference on Learning Representations**. arXiv:1803.07728.

Grill, J.-B. et al. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. **Advances in Neural Information Processing Systems**. arXiv:2006.07733.

Gutmann, M. and Hyvärinen, A. (2010). Noise-Contrastive Estimation: A New Estimation Principle for Unnormalized Statistical Models. **Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics**. PMLR.

He, K. et al. (2020). Momentum Contrast for Unsupervised Visual Representation Learning. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:1911.05722.

He, K. et al. (2022). Masked Autoencoders Are Scalable Vision Learners. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2111.06377.

Ho, J., Jain, A. and Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. **Advances in Neural Information Processing Systems**. arXiv:2006.11239.

Hsu, W.-N. et al. (2021). HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. arXiv:2106.07447.

LeCun, Y., Chopra, S., Hadsell, R., Ranzato, M. and Huang, F. (2006). A Tutorial on Energy-Based Learning. In G. Bakir, T. Hofmann, B. Schölkopf, A. J. Smola and B. Taskar (eds.), **Predicting Structured Data** (pp. 191-246). MIT Press.

Misra, I. and van der Maaten, L. (2020). Self-Supervised Learning of Pretext-Invariant Representations. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. CVF Open Access.

Noroozi, M. and Favaro, P. (2016). Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles. **European Conference on Computer Vision**. arXiv:1603.09246.

Oquab, M. et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193.

Peng, Z. et al. (2022). BEiT v2: Masked Image Modeling with Vector-Quantized Visual Tokenizers. arXiv:2208.06366.

Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI technical report.

Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. **International Conference on Machine Learning**. arXiv:2103.00020.

Siméoni, O. et al. (2025). DINOv3. arXiv:2508.10104.

Song, Y. et al. (2021). Score-Based Generative Modeling through Stochastic Differential Equations. **International Conference on Learning Representations**. arXiv:2011.13456.

Tian, Y., Krishnan, D. and Isola, P. (2020). Contrastive Multiview Coding. **European Conference on Computer Vision**. arXiv:1906.05849.

van den Oord, A., Li, Y. and Vinyals, O. (2018). Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748.

Wang, T. and Isola, P. (2020). Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere. **International Conference on Machine Learning**. arXiv:2005.10242.

Wei, C. et al. (2022). Masked Feature Prediction for Self-Supervised Visual Pre-Training. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2112.09133.

Wu, Z., Xiong, Y., Yu, S. X. and Lin, D. (2018). Unsupervised Feature Learning via Non-Parametric Instance Discrimination. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. CVF Open Access.

Xie, Z. et al. (2022). SimMIM: A Simple Framework for Masked Image Modeling. **IEEE/CVF Conference on Computer Vision and Pattern Recognition**. arXiv:2111.09886.

Zbontar, J. et al. (2021). Barlow Twins: Self-Supervised Learning via Redundancy Reduction. **International Conference on Machine Learning**. arXiv:2103.03230.

Zhai, X. et al. (2023). Sigmoid Loss for Language Image Pre-Training. **IEEE/CVF International Conference on Computer Vision**. arXiv:2303.15343.

Zhang, R., Isola, P. and Efros, A. A. (2016). Colorful Image Colorization. **European Conference on Computer Vision**. arXiv:1603.08511.

Zhou, J. et al. (2022). iBOT: Image BERT Pre-Training with Online Tokenizer. arXiv:2111.07832.